Refinement-DAG Software Development

Abstract

Section titled “Abstract”

Agent-authored software requires a principled boundary between the behavioral guarantees it must preserve and the internal implementation details an automated agent is permitted to change. The development method presented here serves five root purposes: realizing committed behavior, preserving authority over governing meaning, keeping justified change local, reducing the context required for human governance, and keeping assurance accountable to evidence. Its compositional theorems concern finite visible-trace safety, relating local component machines, parent-owned contracts, executable leaves, and architectural connectors through five distinct inclusion roles. Three chosen operational disciplines make system boundaries unambiguous: Restricted Leaf Normal Form (pure, terminating handlers over state-indexed enabled inputs); letter-or-erasure trace observations paired with deterministic protocol monitors; and single-owner stateful runtime instances, allowing sharing without replication only for genuinely state-free definitions. A guarded, execution-witness-indexed wiring obligation bridges local verification to concrete composition, establishing conditional root soundness, the soundness of root-covering inspection frontiers, and an incremental frame theorem for exact-operand evidence reuse.

Separate operational obligations cover required reception and readiness; selected response duties additionally rule out silent loss and boundary-owned non-progress while permitting declared, inspectable external waits. A complete durable-acknowledgement example distinguishes these safety and progress claims and makes their explicit runtime premises transparent. Interactive inspection and context-limited intent authoring are formulated as governance requirements; no empirical reduction in human effort is claimed.

1. Introduction

Section titled “1. Introduction”

When software is authored or modified by autonomous agents, an agent can make a failing test suite pass by repairing the implementation, by weakening test assertions, or by altering the underlying meaning of the requirement. Only the first constitutes a genuine implementation repair. A sound development method must distinguish these actions before a successful tool invocation can be accepted as verification evidence. We adopt a cooperative-fallibility baseline: agents and developers are assumed to act without adversarial malice, but their proposals are fallible, prone to scope creep, and liable to introduce subtle regressions or false abstractions unless checked against formal boundaries.

The same distinction governs system composition. A storage subsystem may satisfy its local component model while that model permits acknowledgement prior to durable persistence. Conversely, a correct implementation may fail refinement simply because its abstraction admits environment executions that cannot occur in deployment. Neither failure mode is resolved by adding reviewers to an informal boundary. The specification, the abstraction, the executable implementation, and the evidence claim must be maintained as distinct, first-class objects.

To establish these boundaries, the development method is organized around five root purposes:

- **P1: Realize committed behavior.** Actual whole-system execution must satisfy accepted promises under their declared operational conditions; required operating conditions cannot be excluded merely to make verification succeed.
- **P2: Preserve authority over governing meaning.** The adoption, modification, and retirement of governing commitments must be authorized by entitled stakeholders or an applicable delegation; implementation authority does not confer authority to rewrite system requirements.
- **P3: Keep justified change local.** When an implementation or environment changes, previously established guarantees and evidence are reused whenever their validity conditions survive. Locality is conditional on evidence applicability, not an unconstrained property of every source edit.
- **P4: Reduce human governance context.** The system representation must minimize the extraneous detail a human must hold in working memory. During upstream intent authoring, stakeholders express or revise intent using decision-relevant responsibilities and interfaces without reconstructing unrelated leaf code or proof scripts. During downstream assessment, interactive visual inspection

exposes commitments, boundaries, changes, unresolved duties, and external dependencies without concealing qualifications that could alter an architectural decision.
- **P5: Keep assurance accountable to evidence.** Every claim must be bound to its identified subject, assumptions, and supporting artifacts; commitments, open questions, empirical observations, and refutable hypotheses remain strictly distinguished.

To realize these purposes, the calculus developed here uses one finite-safety relation—set inclusion of finite visible traces—across five distinct typed roles: satisfaction, wiring adequacy, decomposition, child refinement, and leaf realization. Their operands are not interchangeable. In particular, a parent proves its architectural design against abstract contracts, whereas an executable connector is proved sound for operational witnesses that satisfy those contracts. A development DAG records these obligations and their provenance without allowing obligation activation to depend on proof success.

Systems admitted to this calculus obey three orthogonal operational disciplines:

- **O1: Restricted Leaf Normal Form (RLNF).** A leaf consists of pure, terminating, explicit-state handlers $\delta_\ell \delta\ell$, defined over state-indexed enabled inputs $\mathrm{en}_\ell(s)\mathrm{en}\ell(\mathrm{s})$. Each handler returns a finite list of emitted outputs. Substrate operations are modeled as command events; their asynchronous results arrive as response inputs.
- **O2: Homomorphic observation and deterministic contract monitors.** Each edge observation is induced by a letter-or-erasure action map. A Level-3 contract is a deterministic, single-initial, prefix-closed protocol monitor. These restrictions guarantee an exact equivalence between trace inclusion and relational forward simulation.
- **O3: Single-owner stateful instances / state-free sharing.** Every stateful runtime instance has at most one owning parent boundary, which strictly routes and mediates all client access. Definitions with a single semantic state and no hidden mutable state may be shared across multiple parents in the DAG. In contrast, separate runtime instances of a stateful definition possess distinct runtime identities and disjoint owned mutable state.

These operational principles are chosen design disciplines of the calculus rather than unique mathematical consequences of P1—P5. They establish orthogonal language, observation, and ownership foundations. To prevent unsolicited outputs, hidden shared aliases, or trivial input refusal (such as producing only the empty trace $\varepsilon\epsilon$), the calculus enforces explicit delivery, confinement, relative input receptiveness, and evidence applicability obligations at each system boundary. Furthermore, to eliminate silent hangs and uninspected stalling, selected boundaries enforce duty-relative readiness and non-silent response conditions that permit declared, inspectable external waits without weakening terminal commitments.

The contributions of this work are:

1. The restricted operational semantics and five-place refinement algebra;
2. A guarded, witness-indexed composition theorem;
3. Conditional DAG and frontier soundness theorems;
4. Protected-boundary versioned evolution with exact-operand proof reuse (an incremental frame theorem and interface cutoff corollary);
5. A fully instantiated, end-to-end durable-acknowledgement example with discriminating counterexamples; and
6. A contract-relative non-silent response condition that distinguishes permitted terminal outcomes from genuine external waits, with a separate readiness and progress argument for the worked example.

Sections 2—4 define the finite-safety algebra, the development DAG record, and the separate reception, readiness, and non-silent response obligations. Sections 5—7 address composition, human-governed development protocols, and evidence-preserving evolution. Section 8 details the durable-acknowledgement case study, its safety proof, its scoped progress argument, and discriminating regressions. Sections 9—11 review related work, discuss fundamental limitations, and conclude. Appendix A provides expanded mathematical proofs; Appendix B summarizes formal notation.

2. The Refinement Algebra

Section titled “2. The Refinement Algebra”

2.1 Observations, machines, and action signatures

Section titled “2.1 Observations, machines, and action signatures”

An interface XX has a disjoint action signature

$$\Sigma_X = \Sigma_X^{\mathrm{in}} \uplus \Sigma_X^{\mathrm{out}} \uplus \Sigma_X^{\mathrm{int}}, \qquad \Sigma_X^{\mathrm{vis}} = \Sigma_X^{\mathrm{in}} \uplus \Sigma_X^{\mathrm{out}}.$$
$$\Sigma X = \Sigma X\mathrm{in} \uplus \Sigma X\mathrm{out} \uplus \Sigma X\mathrm{int}, \qquad \Sigma X\mathrm{vis} = \Sigma X\mathrm{in} \uplus \Sigma X\mathrm{out}.$$

A machine is a labeled transition system $M = (S, S_0, \Sigma_X, \rightarrow)$M = (S, S0, ΣX, →) with nonempty initial states $\varnothing \neq S_0 \subseteq S\varnothing$ = S0 ⊆ S. An execution is a finite path starting in S_0S0, including the length-zero path. Its visible trace deletes internal labels. Thus

$\mathrm{tr}(M) = \{\mathrm{vis}(\rho) \mid \rho \text{ is a finite execution of } M\} \subseteq (\Sigma_X^{\mathrm{vis}})^*$.tr(M) = {vis(ρ) | ρ is a finite execution of M} ⊆ (ΣXvis)∗.

Write ϵϵ for the empty word, $u \preceq t$u ⪯ t for word-prefix order, and $\mathrm{pref} L = \{u \mid \exists t \in L,\ u \preceq t\}$pref L = {u | ∃t ∈ L, u ⪯ t} for prefix closure. A specification is an arbitrary language $\Phi \subseteq (\Sigma_X^{\mathrm{vis}})^*$Φ ⊆ (ΣXvis)∗, whereas every operational machine trace language is non-empty, contains ϵϵ, and is closed under prefixes.

We adopt the action partition of Lynch and Tuttle (1989), but do not assume their global input-enabledness. Our open-interface admission rule is relative to a declared protocol (§2.5). Denotational behavior sets QQ and operational execution witnesses NN are kept distinct: $Q = \mathrm{tr}(N)$Q = tr(N) abstracts away state spaces, input acceptance, ownership, and runtime scheduling structure. Concrete composition plugs in operational witnesses NN, rather than unstructured sets of words.

Finite visible traces record which finite sequences of visible events can occur. They do not record refusal, whether an execution terminates or continues forever, or whether a duty remains permanently pending. Operational obligations introduced below evaluate the operational execution witness NN and its declared runtime interpretation directly, rather than attempting to recover liveness, readiness, or divergence distinctions from the trace language.

2.2 Satisfaction, refinement, homomorphic edge abstraction, and contract precision

Section titled “2.2 Satisfaction, refinement, homomorphic edge abstraction, and contract precision”

Satisfaction and same-interface refinement are defined by set inclusion:

$$M \vDash \Phi \iff \mathrm{tr}(M) \subseteq \Phi, \qquad M \preceq M' \iff \mathrm{tr}(M) \subseteq \mathrm{tr}(M').$$
$$\mathrm{M} \vDash \Phi \iff \mathrm{tr}(\mathrm{M}) \subseteq \Phi, \qquad \mathrm{M} \preceq \mathrm{M}' \iff \mathrm{tr}(\mathrm{M}) \subseteq \mathrm{tr}(\mathrm{M}').$$

For an edge (v, c)(v, c), the child component and parent slot may have different visible alphabets. Fix a letter-or-erasure map:

$$\lambda_{v,c} : \Sigma_c^{\mathrm{vis}} \longrightarrow \Sigma_{v,c}^{\mathrm{vis}} \cup \{\epsilon\}, \qquad \alpha_{v,c} = \lambda_{v,c}^* = \mathrm{mapTrace}(\lambda_{v,c}).$$
$$\lambda \mathrm{v,c} : \Sigma \mathrm{cvis} \longrightarrow \Sigma \mathrm{v,cvis} \cup \{\epsilon\}, \qquad \alpha \mathrm{v,c} = \lambda \mathrm{v,c}* = \mathrm{mapTrace}(\lambda \mathrm{v,c}).$$

The map $\lambda_{v,c}^*$λv,c∗ extends to words by concatenation, sending ϵϵ to ϵϵ. On trace languages it acts by direct image. Inputs and outputs that remain visible retain their declared roles at the parent slot; erased inputs are still checked against the child-domain input scope. Internal child actions are likewise erased when matching transitions. All visible abstractions are therefore uniformly induced by the designated homomorphism λ^*λ∗.

Lemma (Prefix Commutation of Letter-or-Erasure Homomorphisms). For every language LL,

$\lambda^*(\mathrm{pref} L) = \mathrm{pref}(\lambda^* L)$.λ∗(pref L) = pref(λ∗L).

Proof. The image of any source prefix is directly a target prefix. Conversely, given any prefix $p \preceq \lambda^*(t)$p ⪯ λ∗(t) for some $t \in L$t ∈ L, scanning the letters of tt accumulates non-erased target letters one at a time (or zero if erased); because each source letter contributes at most one letter to the target, some minimal prefix of tt maps exactly to pp. The empty prefix is witnessed by ϵϵ. Taking the union over all words in LL completes the proof for any language LL, including $L = \varnothing$L = ∅. □□

The restriction to letter-or-erasure maps is essential: for $L = \{a\}$L = {a} and a multi-letter substitution $\lambda(a) = xy$λ(a) = xy, the left side is $\lambda^*(\mathrm{pref}\{a\}) = \{\epsilon, xy\}$λ∗(pref{a}) = {ϵ, xy}, whereas the right side is $\mathrm{pref}(\lambda^*\{a\}) = \{\epsilon, x, xy\}$pref(λ∗{a}) = {ϵ, x, xy}.

Let π_{in}πin denote the projection that erases every non-input visible event. An admissible input language $A \subseteq (\Sigma_c^{\text{in}})^*$A ⊆ (Σcin)∗ is prefix-closed and contains ϵϵ. Define the input-restricted language:

$Q \restriction_A = \{t \in Q \mid \pi_{\text{in}}(t) \in A\}$.Q ↾A= {t ∈ Q | πin(t) ∈ A}.

Its operational realization $\text{Restrict}_A(N)$RestrictA(N) has composite states (s, h)(s, h), initial states (s_0, ϵ)(s0, ϵ), and the following transitions. A source input step $s \xrightarrow{i} s'$s i

---

s′ is retained precisely when $hi \in A$hi ∈ A, producing (s', hi)(s′, hi). Every source output or internal transition is retained and leaves the input history hh unchanged. No output is filtered.

Restriction equality.

$$\text{tr}(\text{Restrict}_A(N)) = \text{tr}(N) \restriction_A .$$
$$\text{tr}(\text{RestrictA(N)}) = \text{tr(N)} \restriction \text{A} .$$

Proof. Projecting an execution of $\text{Restrict}_A(N)$RestrictA(N) to NN yields an execution of NN whose history component records its input projection and remains in AA. Conversely, given an execution of NN whose final input history is in AA, prefix closure of AA ensures that every intermediate input history belongs to AA, so every step lifts to $\text{Restrict}_A(N)$RestrictA(N). Initial states remain nonempty. □□

The canonical edge transformer is

$T_{v,c}(Q) = \lambda^*_{v,c}(Q \restriction_{A_{v,c}})$.Tv,c(Q) = λv,c∗(Q ↾Av,c).

Because restriction and direct image preserve set inclusion, $T_{v,c}$Tv,c is monotone. For a slot language $L_{v,c}$Lv,c, edge refinement is $T_{v,c}(\text{tr}(M_c)) \subseteq L_{v,c}$Tv,c(tr(Mc)) ⊆ Lv,c. Equivalently,

$$\text{tr}(M_c) \restriction_{A_{v,c}} \subseteq (\lambda^*_{v,c})^{-1}(L_{v,c}).$$
$$\text{tr(Mc)} \restriction \text{Av,c} \subseteq (\lambda\text{v,c}*)\text{—1(Lv,c)}.$$

Inverse-image notation is deliberately distinct from the star denoting word extension.

The calculus admits four levels of contract precision. They describe choices of predicates, not four different refinement relations.

Level	Contract meaning	What evidence must establish
0: Top T⊤	Every well-typed finite trace	Inclusion is trivial; no nontrivial protocol guarantee follows.
1: Type/shape	Every recorded value or event has the required shape	A sound typing or schema argument for the selected observation; a compiler invocation alone is insufficient without its soundness and language assumptions.
2: Pre/post	For completed calls, $P(x) \Rightarrow Q(x, y)$P(x) ⇒ Q(x, y); behavior outside PP is unconstrained	A sound Hoare-style argument. A completed-call relation alone does not establish termination, initialization, input acceptance, or feasibility.
3: Protocol	A deterministic partial monitor admitting all prefixes of its runs	A trace-inclusion proof or the initialized simulation of §4.3, together with separate admission checks.

Levels 1 and 2 lift directly to trace predicates by evaluating completed request–response pairs. Following the feasibility foundations of Unifying Theories of Programming (Woodcock and Cavalcanti, 2004), specifications require explicit operational witnesses to ensure that preconditions are satisfiable and models are non-empty. In this setting, RLNF establishes totality and termination for each enabled handler over its declared enabled state–input domain. Environmental feasibility and the satisfiability of required uses are verified separately; an uninvoked partial branch with unsatisfiable preconditions ($P = \text{false}$P = false) remains a valid partial-correctness specification for unused operations.

These precision levels describe safety meanings and their associated verification evidence. None of these levels alone implies required input reception, operational readiness, or eventual response. A monitor encoding may represent a coarser safety predicate without altering its language; neither an artificial level tag nor an independent verifier is mathematically required.

2.3 Connectors

Section titled “2.3 Connectors”

A connector skeleton W_vWv has legal finite joint trajectories $\mathrm{legal}_{W_v}(r)$legalWv(r), slot slices $\mathrm{slice}_c(r)$slicec(r), and a parent observation $\mathrm{obs}_v(r)$obsv(r). Its denotation is

$$K_v((L_c)_c) = \{\mathrm{obs}_v(r) \mid \mathrm{legal}_{W_v}(r) \wedge \forall c,\ \mathrm{slice}_c(r) \in L_c\}.$$

Kv((Lc)c) = {obsv(r) | legalWv(r) ∧ ∀c, slicec(r) ∈ Lc}.

A trajectory retains the correlations needed for routing, synchronization, and any declared substrate behavior. Legality is fixed independently of the argument languages. Its finite prefixes are legal, slices of trajectory prefixes are prefixes of the original slices, and every prefix of $\mathrm{obs}_v(r)$obsv(r) is the observation of some trajectory prefix of rr. Letter–or–erasure observation of a trajectory’s individual events supplies this last, prefix–lifting property by §2.2; prefix preservation alone would not suffice. Consequently, K_vKv maps prefix–closed slot languages to a prefix–closed parent language: any prefix of an observed trajectory lifts to a valid trajectory prefix whose slices remain within the admitted prefix languages, with initial states established by the operational realization.

Lemma 1 (Monotonicity). If $L_c \subseteq L_c'$Lc ⊆ Lc′ for every slot, then $K_v((L_c)_c) \subseteq K_v((L_c')_c)$Kv((Lc)c) ⊆ Kv((Lc′)c).

Proof. Keep the same legal trajectory witnessing membership on the left; every slice also belongs to its enlarged language. □□

The concrete connector $C_v[N]$Cv[N] provides the operational composition of component witnesses, specifying explicit transition, routing, and instance allocation rules. The bridge between this operational composition and the denotational transformer K_vKv is established by the guarded wiring obligation in Condition 5 (§3.4).

2.4 The finite–safety relation and its five uses

Section titled “2.4 The finite–safety relation and its five uses”

For Level–3 slots write $L_{v,c} = \mathrm{tr}(B_{v,c})$Lv,c = tr(Bv,c); coarser slots supply their declared trace predicates directly.

Family	Behavioral obligation
$\mathrm{Sat}(v, j)$Sat(v, j)	$\mathrm{tr}(M_v) \subseteq [\![j]\!]$tr(Mv) ⊆ ⟦j⟧ for an active owned intent
$\mathrm{Wire}(v)$Wire(v)	Guarded universal delivery and $\mathrm{tr}(C_v[N]) \subseteq K_v((T_{v,c}(\mathrm{tr}(N_c)))_c)$tr(Cv[N]) ⊆ Kv((Tv,c(tr(Nc)))c)
$\mathrm{Dec}(v)$Dec(v)	$K_v((L_{v,c})_c) \subseteq \mathrm{tr}(M_v)$Kv((Lv,c)c) ⊆ tr(Mv)
$\mathrm{Ref}(v, c)$Ref(v, c)	$T_{v,c}(\mathrm{tr}(M_c)) \subseteq L_{v,c}$Tv,c(tr(Mc)) ⊆ Lv,c
$\mathrm{Impl}(\ell)$Impl(ℓ)	$\mathrm{tr}(N_\ell) \subseteq \mathrm{tr}(M_\ell)$tr(Nℓ) ⊆ tr(Mℓ) for the scoped executable witness

The five obligations form a coherent algebraic inclusion chain, establishing compositional finite–safety properties. Operational delivery and admission supply its essential execution premises. Required input reception, operational readiness, and scoped non–silent response are additional behavioral properties of the admitted execution witnesses; they are not consequences of language inclusion alone.

2.5 Semantic profile scope, reception, readiness, and non–silent response

Section titled “2.5 Semantic profile scope, reception, readiness, and non–silent response”

The finite–safety profile Γ_{safe}Γsafe fixes finite visible traces, prefix–closed inclusion, the action partition, and operational admission, ensuring that initialization is non–empty, state invariants hold, and accepted environmental contexts admit feasible physical realizations.

To eliminate silent drops, uninspected hangs, and enabled–but–untaken continuations, boundaries with response commitments enforce three additional operational obligations: relative input reception, operational readiness, and contract–relative non–silent response.

2.5.1 Relative input reception is configuration–sensitive

Section titled “2.5.1 Relative input reception is configuration–sensitive”

Relative input receptiveness is checked across reachable concrete configurations, not merely by examining visible trace words. For a deterministic visible protocol monitor BB and a concrete finite execution ρρ of witness NN, let

$$s = \mathrm{last}(\rho), \qquad h = \pi_{\mathrm{in}}(\mathrm{vis}(\rho)), \qquad q = \hat{\beta}(q_0, \lambda^*(\mathrm{vis}(\rho))).$$

s = last(ρ), h = πin(vis(ρ)), q = β

At **every** reachable in–scope configuration and history (s, h)(s, h), each protocol–permitted concrete input ii satisfying $hi \in A$hi ∈ A must have an enabled acceptance transition $s \xrightarrow{i} s'$s i

---

s′ in the concrete machine.

When inputs are mapped or erased under homomorphism λλ, the boundary declaration specifies their concrete acceptance relation; hidden response protocols remain explicit. A pending–output state cannot waive a permitted input through an arbitrary internal readiness flag. Either the corresponding protocol state forbids the input, or a modeled input arrival buffer must accept it. Two executions with identical visible traces may reach different internal configurations; relative reception requires acceptance at all reachable configurations corresponding to that protocol state.

2.5.2 Duties, permitted outcomes, and genuine external waits

Section titled “2.5.2 Duties, permitted outcomes, and genuine external waits”

For each boundary carrying an accepted response commitment, an accepted request occurrence opens a duty dd. Duties are tracked by occurrence identity, avoiding the need for bureaucratic authorization tokens. Evaluated over the concrete runtime configuration enriched by necessary correlation history:

- P_dPd: the duty is outstanding. It becomes true upon request acceptance and remains true until a permitted terminal outcome for that occurrence is observed at the promising boundary.
- D_dDd: a permitted terminal outcome for duty dd is emitted at the promising boundary. Success and failure outcomes count toward D_dDd only under their contractually declared conditions. Emitting an internal outcome at an intermediate child boundary does not discharge the parent’s duty.
- $W_{d,e}$Wd,e: duty dd is genuinely blocked on an unresolved external prerequisite occurrence or condition ee, and the wait is accurately inspectable. $W_{d,e}$Wd,e strictly implies P_dPd.

The predicate $W_{d,e}$Wd,e is a factual causal condition, not an arbitrary state annotation:

1. **External prerequisite:** ee is an actual outstanding event or condition outside the realization and scheduling authority of this boundary.
2. **Causal dependency:** Under the accepted contract and current conditions, the next step necessary to advance duty dd depends on e e, and no boundary–owned action is presently due instead. An already available result, an undrained pending output, a required failure diagnostic, or a runnable internal continuation cannot be concealed as an external wait.
3. **Accurate inspectability:** The identity of duty dd, the responsible boundary, the external prerequisite ee, and known status/uncertainty are exposed for external or human inspection. A specialized UI schema or root–cause verifier is not mandated.
4. **Resolution invalidation:** Once ee resolves, $W_{d,e}$Wd,e ceases to hold. A subsequent retry or new external request creates a distinct prerequisite occurrence and cannot retroactively reuse an old wait explanation.

At a composite parent boundary, all child components, internal connectors, and owned scheduling queues lie inside the realization. Consequently, a child waiting for a sibling component is an internal dependency of the parent, not a parent external wait. A parent may

expose a genuine external prerequisite reached through its children (such as an unresolved storage flush), but internal sibling ping-pong remains owned work.

The interpretation of duties and permitted outcomes derives entirely from the accepted contract. It is not inferred from private state names such as `Done`, nor from whether an internal handler returns an empty output list. Pure input-only operations that carry no response promise define no response duty.

Each accepted contract declares a wait policy E_d^{∞}Ed∞, representing the set of external prerequisite occurrences or conditions at which duty dd is permitted to remain pending indefinitely. For a strict terminal-response contract, $E_d^{\infty} = \varnothing$Ed∞ = ∅. The predicate $W_{d,e}$Wd,e may hold during legitimate **transient** waiting for such a duty; genuine waiting does not itself grant permission to hang indefinitely. Conversely, permission to wait indefinitely does not fabricate a prerequisite where none exists.

2.5.3 Operational readiness (ReadyReady)

Section titled “2.5.3 Operational readiness (Ready\mathrm{Ready}Ready)”

Let the duty-relative transition relation be defined as a restriction of the actual transition relation:

$$s \xrightarrow{\mathrm{owned}(d)} s' \iff \exists a.\ s \xrightarrow{a} s' \ \wedge\ \mathrm{Owned}_{\upsilon}(s,a,s') \ \wedge\ \mathrm{Continuation}_{d}(s,a,s').$$

s owned(d)

Here $\mathrm{Owned}_{\upsilon}$Ownedv classifies internal computation, dispatch, routing, or output steps belonging to boundary υv, and $\mathrm{Continuation}_{d}$ Continuationd asserts that the transition advances duty dd along an admitted path. This classification must be justified from the actual implementation witness; it cannot be asserted by fiat or treat unrelated background activity as progress toward dd.

Required readiness is:

$$\forall s \text{ reachable},\ d.\quad P_d(s) \wedge \neg\exists e\, W_{d,e}(s) \implies \exists s'.\ s \xrightarrow{\mathrm{owned}(d)} s'. \qquad (\mathrm{Ready})$$

∀s reachable, d. Pd(s) ∧ ¬∃e Wd,e(s) ⟹ ∃s′. s owned(d)

Readiness ensures that whenever a duty is outstanding and not blocked on an inspectable external prerequisite, an owned continuation step is enabled. Intermediate internal computation is permitted; the formula does not demand that every pending state immediately emit a terminal output.

2.5.4 Complete-execution non-silent response (NSNS)

Section titled “2.5.4 Complete-execution non-silent response (NS\mathrm{NS}NS)”

Let $\mathrm{CExec}_{\upsilon}(N_{\upsilon},\omega)$CExecv(Nv,ω) denote the maximal executions of the declared runtime semantics at boundary υv, subject to stated execution premises. The runtime semantics accounts for internal computation, scheduling, delay steps, external waiting, and quiescence. Finite maximal (quiescent) executions are extended by idle stuttering for temporal evaluation; infinite executions are maximal by definition. Arbitrary truncated observation prefixes are not complete executions.

For every duty dd committed by the contract:

$$\mathrm{NS}_{\upsilon}(N_{\upsilon}) \iff \forall \pi \in \mathrm{CExec}_{\upsilon}(N_{\upsilon},\omega),\ \forall d.\quad \pi \vDash G\big(P_d \Rightarrow \big[FD_d \vee \exists e \in E_d^{\infty}.FGW_{d,e}\big]\big). \qquad (\mathrm{NS})$$

NSv(Nv) ⟺ ∀π ∈ CExecv(Nv,ω), ∀d. π ⊨ G(Pd ⇒ [FDd ∨ ∃e ∈ Ed∞.FGWd,e]).(NS)

Here FF denotes eventually and GG denotes globally along execution path ππ. The second disjunct requires that the execution eventually settle into a persistent genuine external wait ee that belongs to the contract-permitted set E_d^{∞}Ed∞. In this selected profile, placing $\exists e \in E_d^{\infty}$∃e ∈ Ed∞ outside FGFG requires an unresolved execution to stabilize at one specific external prerequisite instance rather than

cycling among transient waits; services whose contracts admit unbounded external retry polling require a corresponding multi-wait progress profile.

The consequences of NSNS are precise:

- A permitted terminal outcome discharges the duty. A persistent external wait does **not** discharge it; rather, the accepted contract permits that unresolved execution only at a declared member of E_d^∞Ed∞.
- An implementation that accepts a request and transitions to `Done` with an empty output list without emitting a required outcome or entering an external wait fails both ReadyReady and NSNS.
- An execution trapped in an internal loop while a terminal outcome remains enabled fails NSNS, even if safety, reception, and readiness hold.
- A permanently unresolved external prerequisite (such as an indefinite disk flush), accurately inspectable and with no withheld owned work, satisfies NSNS if and only if $e \in E_d^\infty$e ∈ Ed∞.
- For a strict contract ($E_d^\infty = \varnothing$Ed∞ = ∅), transient external waiting is permitted by ReadyReady, but NSNS demands FD_dFDd; if the external result never arrives, NSNS fails. Inspectability does not excuse failure to deliver a promised terminal response.
- Once an external prerequisite resolves, lingering at the old wait state or starving subsequent output drains violates NSNS.
- Periodic cycling through transient wait labels without permanently remaining at an unresolved prerequisite fails the second disjunct $FGW_{d,e}$FGWd,e.

Operational obligations ReadyReady and NSNS apply where response duties are explicitly declared. A safety-only boundary is not silently upgraded to a terminal-response service, and finite prefixes stopping after request reception remain valid safety observations.

2.6 Boundary discipline

Section titled “2.6 Boundary discipline”

Component contracts specify visible interactions—enabled inputs, emitted outputs, response duties, permitted failure conditions, acknowledgements, causal ordering, and client preconditions—while preserving encapsulation of internal algorithms, data structures, and execution policies. A contract may specify which external prerequisites are admitted as persistent waits (E_d^∞Ed∞) and what ordering or acknowledgement duties the caller must satisfy. It must not prescribe a consumer's unrelated internal policies or scheduling strategies.

The parent boundary defines the slot contract, and the child component establishes compliance through its chosen internal realization. RLNF serves as an operational admission standard for verified handlers, guaranteeing termination and explicit state transitions on enabled inputs without constraining unrelated component design choices. Implementation-specific proof artifacts, such as internal inductive ranks or termination measures, are evidence supplied by a component, not obligations imposed on caller contracts.

3. The Refinement DAG and Soundness

Section titled “3. The Refinement DAG and Soundness”

3.1 The development DAG record

Section titled “3.1 The development DAG record”

A development record is

$$G = (V, M, \chi, I, J, E).$$
G = (V , M, χ, I, J, E).

VV is a finite set of cells. $M(v)$M(v) specifies the interface, local machine M_vMv, semantic profile (including declared response duties and wait policies), and typed context references. An internal cell's $\chi(v)$χ(v) records C_v, W_v, K_vCv, Wv, Kv, its child slots, and $(B_{v,c}, \lambda_{v,c}, A_{v,c})_c$ (Bv,c, λv,c, Av,c)c; a leaf's $\chi(\ell)$χ(ℓ) records its RLNF realization, shell bindings, and runtime interpretation. The edge relation $v \to c$v → c is acyclic. Write $\mathrm{ch}(v)$ch(v), $\mathrm{par}(v)$par(v), V_{root}Vroot, and V_{leaf}Vleaf for children, parents, roots, and leaves.

The development record explicitly decouples obligation definitions from their verification status: II tracks formal intents with stable identities, unique owners, scoped predicates $[\![j]\!]$[[j]], and authority tags (`root`, `protected`, or `delegated`). The support mapping $J(j)$J(j) defines finite collections of conjunctive support groups, where alternative groups represent disjunctive proof paths over an acyclic support–reference graph. The evidence store EE maps typed proof obligations to formal verification artifacts, keeping obligation activation independent of evidence presence. Operational duty and execution interpretations are maintained directly within the semantic profile of $M(v)$M(v), avoiding redundant parallel ledgers. Stateful runtime instances and stateless reusable definitions are partitioned according to the single-owner discipline of §3.5.

3.2 Active intents and positive support evaluation

Section titled “3.2 Active intents and positive support evaluation”

Define the monotone support operator $F(X) = I^{\text{root}} \cup \{j \mid \exists P \in J(j),\ P \subseteq X\}$F(X) = Iroot ∪ {j ∣ ∃P ∈ J(j), P ⊆ X}, and let $A = \mu X.F(X)$A = μX.F(X) denote its unique least fixed point. Because FF operates over finite conjunctive support groups without negation, and Condition 1 restricts (I, J)(I, J) to a finite support graph, the Knaster–Tarski fixed-point theorem ensures the existence of AA, which is computed by finite iteration starting from $\varnothing$∅; acyclicity of the support–reference graph further enables an efficient single topological evaluation. An explicitly accepted empty support group activates its conclusion directly. An unsupported cycle, if admitted in a more general support graph, would not bootstrap itself in the least fixed point.

Write $A(v) = \{j \in A \mid \text{owner}(j) = v\}$A(v) = {j ∈ A ∣ owner(j) = v} and

$\Phi(v) = \bigcap_{j \in A(v)} [\![j]\!]$, Φ(v) = ∩j∈A(v)[[j]],

with empty intersection equal to the full typed trace space. Each active intent has exactly one owner.

Lemma 2 (Evidence Separation). AA depends only on I, JI, J, not on EE.

Proof. FF does not mention EE, so neither do its iterates or least fixed point. A failed or absent evidence record cannot deactivate its obligation. □□

Conjunction within a support group and disjunction between alternative groups determine intent activation. The authored support groups explain why an obligation is active within the development record; they neither prove its behavioral correctness nor require an exhaustive enumeration of all possible minimal supports. Each active behavioral claim must still be discharged by applicable verification evidence.

3.3 Context premises, dependency closure, and staleness

Section titled “3.3 Context premises, dependency closure, and staleness”

All scoped judgments are interpreted over a common world $\omega \in \Omega$ω ∈ Ω. A hypothesis reference hh has a fixed meaning $\text{Meaning}(h, \omega)$Meaning(h, ω). An evidence claim has the form $\forall \omega,\ H_q(\omega) \Rightarrow q(\omega)$∀ω, H_q(ω) ⇒ q(ω). Context references include environmental assumptions, runtime interpretations, proof-method soundness premises, and their own prerequisite references.

For a root rr, $\text{Deps}(r)$Deps(r) is the transitive closure of every hypothesis used by its realization, admission, edge, wiring, decomposition, satisfaction, and evidence arguments. In particular, it is not merely the set of descendant cells. When operational claims are made, $\text{Deps}(r)$Deps(r) additionally includes reception and readiness assumptions, duty interpretations, complete-execution semantics, external-wait interpretations, scheduler premises, and any relied-on external response premises. In particular, for the worked example of §8, the eventual-write premise H_{write}Hwrite belongs to $\text{Deps}(r)$Deps(r) and the operational context H_r^{op}Hrop as an explicit conditional environmental premise; the finite-safety proof does not use it, and the shell graph does not prove it. Define

$H_r(\omega) = \bigwedge_{h \in \text{Deps}(r)} \text{Meaning}(h, \omega)$.Hr(ω) = ∧h∈Deps(r) Meaning(h, ω).

Each local context used in the root proof must be entailed by H_rHr. When reusing evidence previously established under context H_{old}Hold in a new context H_{new}Hnew, applicability requires the explicit entailment $H_{\text{new}} \Rightarrow H_{\text{old}}$Hnew ⇒ Hold alongside identical typed claim correspondence, maintaining a rigorous distinction between formal logical implication, environmental feasibility, and empirical substrate premises. Declaring a dependency list does not automatically prove its own completeness; modifications to a hypothesis or operand

propagate across semantic dependency edges, marking downstream obligations as pending until applicability is re-verified or new evidence is supplied. Historical evidence artifacts and prior derivation records are preserved for auditability.

3.4 Intra-DAG validity and guarded witness-indexed wiring

Section titled “3.4 Intra-DAG validity and guarded witness-indexed wiring”

Let $N_v(\omega)$Nv(ω) be the actual scoped execution witness of cell vv: a leaf realization or the composition $C_v[(N_c)_c]$Cv[(Nc)c]. Put $\mathrm{Beh}_{\mathrm{exec}}(v,\omega) = \mathrm{tr}(N_v(\omega))$Behexec(v,ω) = tr(Nv(ω)). A leaf scope restricts environment inputs before observation. Every use inside a parent must justify that scope by delivery; the root’s external input scope is an explicit premise in H_rHr.

The following conditions define intra-DAG validity for finite safety, pointwise under the applicable contexts.

1. **Structure and support.** The child and support graphs are finite and acyclic; interfaces, intent ownership, active supports, and runtime identities are well formed.
2. **Context.** All used premises and their transitive dependencies are declared; their interpretation and scope transports are fixed. Feasibility and unresolved empirical assumptions are reported separately from implication proofs.
3. **Satisfaction.** Every active owned intent has applicable evidence for $\mathrm{Sat}(v,j)$Sat(v,j).
4. **Decomposition.** Every internal cell has applicable evidence for $\mathrm{Dec}(v)$Dec(v), with the trajectory transformer of §2.3.
5. **Operational realization.** Every leaf has applicable ImplImpl evidence. Every internal cell has the guarded universal $\mathrm{Wire}(v)$Wire(v) certificate below.
6. **Admission and edges.** Every actual witness satisfies its declared Adm_vAdmv, including initialization, relative reception, runtime confinement, and interface interpretation. Every edge satisfies $\mathrm{Ref}(v,c)$Ref(v,c) and the ownership discipline of §3.5.

For Level-3 slots, Condition 5 is precisely

$$\forall (N_c)_c.\quad \Big(\forall c,\ \mathrm{Adm}_c(N_c) \wedge \lambda^{*}_{v,c}(\mathrm{tr}(N_c)\restriction_{A_{v,c}}) \subseteq \mathrm{tr}(B_{v,c})\Big) \Longrightarrow \Big(\mathrm{Delivery}_v(C_v,N,A) \wedge \mathrm{tr}(C_v[N]) \subseteq K_v((\lambda^{*}_{v,c}(\mathrm{tr}(N_c)\restriction_{A_{v,c}}))_c)\Big).$$

$$\forall (\mathrm{Nc})\mathrm{c}.\quad (\forall \mathrm{c},\ \mathrm{Admc}(\mathrm{Nc}) \wedge \lambda \mathrm{v},\mathrm{c}*(\mathrm{tr}(\mathrm{Nc}) \restriction \mathrm{Av},\mathrm{c}) \subseteq \mathrm{tr}(\mathrm{Bv},\mathrm{c})) \Longrightarrow (\mathrm{Deliveryv}(\mathrm{Cv},\mathrm{N},\mathrm{A}) \wedge \mathrm{tr}(\mathrm{Cv}[\mathrm{N}]) \subseteq \mathrm{Kv}((\lambda \mathrm{v},\mathrm{c}*(\mathrm{tr}(\mathrm{Nc})$$

The quantified objects are typed execution witnesses, not arbitrary trace sets. Adm_cAdmc is a fixed boundary-domain predicate, not a test for the particular implementation’s identity. For coarse slots substitute their declared language for $\mathrm{tr}(B_{v,c})$tr(Bv,c).

$\mathrm{Delivery}_v$Deliveryv means that every concrete finite composite execution has one legal shared trajectory, with child execution slices from the very witnesses supplied to C_vCv, the same parent observation, and each child’s actual input history in $A_{v,c}$Av,c. Its slot slice is $\lambda^{*}_{v,c}$ λv,c∗ of that child’s visible trace. By the definition of K_vKv, the existence of this shared trajectory directly entails the displayed trace inclusion $\mathrm{tr}(C_v[N]) \subseteq K_v((\lambda^{*}_{v,c}(\mathrm{tr}(N_c)\restriction_{A_{v,c}}))_c)$tr(Cv[N]) ⊆ Kv((λv,c∗(tr(Nc) ↾ Av,c))c); displaying both clauses makes operational delivery and algebraic inclusion explicit without requiring duplicate proofs. Delivery also establishes compatibility with intrinsic leaf input scopes: when scoped witnesses are analyzed under input restriction, the delivery proof demonstrates that the unrestricted operational composition naturally generates only in-scope inputs, ensuring that safety holds without artificially filtering invalid transitions. An inductive first-invalid-input argument exemplifies this derivation (§8.6).

Guarding universal wiring by slot conformance is essential: it guarantees that internal component behaviors, such as premature or spontaneous output emissions, are excluded before composing with the parent connector. The universal wiring certificate is proved once over its abstract admission domain and instantiated whenever concrete children satisfy their respective slot contracts.

Conditions 1–6 define intra-DAG validity for finite safety. Acceptance for a boundary with the selected non-silent response scope additionally requires applicable evidence for its ReadyReady and NSNS obligations on the actual scoped execution witness, alongside verification that the external wait interpretation is faithful and inspectable. These operational obligations may be discharged directly on the composite witness or transported via an applicable property-preserving result; Conditions 1–6 alone establish only the finite-safety inclusion of Theorem 1.

3.5 Edge refinement and the single-owner discipline

Section titled “3.5 Edge refinement and the single-owner discipline”

Every edge satisfies

$T_{v,c}(\mathrm{tr}(M_c)) \subseteq \mathrm{tr}(B_{v,c})$, Tv,c(tr(Mc)) ⊆ tr(Bv,c),

or the corresponding coarse slot predicate. Incoming edges to a shared definition have separate abstractions, input scopes, and contracts; each must be checked.

For a stateful runtime instance cc, $|\mathrm{par}(c)| \le 1$| par(c)| ⩽ 1. All access crosses its owning parent connector, and there is no cross-branch hidden alias to its state or mutable substrate. A reusable stateful definition may be instantiated more than once, but its instances have distinct identities and disjoint owned mutable state. A definition shared without replication is state-free only if $|S_c| = 1$|Sc| = 1 and its complete semantic closure, including descendants and substrate dependencies, has no mutable state. A composite owning a stateful descendant is therefore stateful for this rule. An effectful wrapper around a shared connection is not state-free merely because its source-level object has no fields. Runtime queues and pending-output buffers also have owners; sharing a pure handler definition does not share its invocation buffers.

Single-Parent Enclosure Lemma. Under this ownership and access discipline, every interaction with a stateful instance is enclosed by its unique parent boundary (or its external root boundary), and its ownership ancestry is a chain.

Proof. A non-root instance has one immediate owner. Every legal access crosses that owner by the access discipline. Repeating the owner relation yields at most one next ancestor at each step; acyclicity makes the sequence finite. Thus two purported independent owner paths cannot diverge above the instance, and an interaction from elsewhere must enter through the owning boundary rather than an alias. □□

The Single-Parent Enclosure Lemma establishes the exact architectural boundary where aggregate client interactions must be verified, ensuring that all access paths are mediated by the owning parent; it locates the boundary responsible for interactions rather than proving absence of deadlock or eventual progress by itself.

3.6 Theorem 1: Refinement-DAG finite-safety soundness

Section titled “3.6 Theorem 1: Refinement-DAG finite-safety soundness”

Theorem 1 (Refinement-DAG Finite-Safety Soundness). Let GG satisfy Conditions 1–6, with typed edge maps, declared scope transports, and all obligation evidence applicable under the full dependency contexts. For every root rr,

$$\forall \omega, \quad H_r(\omega) \Longrightarrow \mathrm{Beh}_{\mathrm{exec}}(r,\omega) \subseteq \Phi(r,\omega).$$
∀ω, Hr(ω) ⟹ Behexec(r, ω) ⊆ Φ(r, ω).

The analogous conditional inclusion into $\mathrm{tr}(M_v)$tr(Mv) and each active owned intent holds for every cell under its dependency closure.

Proof. Fix ωω satisfying H_rHr. All local contexts in the proof of rr follow by dependency closure. Use well-founded induction on the child relation. At a leaf, ImplImpl gives $\mathrm{tr}(N_\ell) \subseteq \mathrm{tr}(M_\ell)$tr(Nℓ) ⊆ tr(Mℓ). At an internal vv, the induction hypotheses and monotonicity of $T_{v,c}$Tv,c give

$T_{v,c}(\mathrm{tr}(N_c)) \subseteq T_{v,c}(\mathrm{tr}(M_c)) \subseteq \mathrm{tr}(B_{v,c})$.Tv,c(tr(Nc)) ⊆ Tv,c(tr(Mc)) ⊆ tr(Bv,c).

Together with actual-witness admission, these are exactly the guard of $\mathrm{Wire}(v)$Wire(v). Instantiate it, then use Lemma 1 and $\mathrm{Dec}(v)$ Dec(v):

$$\mathrm{tr}(N_v) \subseteq K_v((T_{v,c}(\mathrm{tr}(N_c)))_c) \subseteq K_v((\mathrm{tr}(B_{v,c}))_c) \subseteq \mathrm{tr}(M_v).$$
tr(Nv) ⊆ Kv((Tv,c(tr(Nc)))c) ⊆ Kv((tr(Bv,c))c) ⊆ tr(Mv).

Satisfaction at vv yields inclusion in every active owned predicate and thus in their intersection. Applying this inductively to root rr completes the proof, with every delivery premise strictly grounded by verified child slot guards. □□

Where the selected non-silent response scope applies, the accepted root claim is the conjunction of this finite trace inclusion with separately established reception, readiness, and non-silent response properties:

$$H_r^{\mathrm{safe}} \wedge H_r^{\mathrm{op}} \implies \big(\mathrm{tr}(N_r) \subseteq \Phi(r)\big) \wedge \mathrm{Rec}_r(N_r) \wedge \mathrm{Ready}_r(N_r) \wedge \mathrm{NS}_r(N_r).$$

Hrsafe ∧ Hrop ⟹ (tr(Nr) ⊆ Φ(r)) ∧ Recr(Nr) ∧ Readyr(Nr) ∧ NSr(Nr).

Each operational conjunct must be supported by its own applicable evidence; H_r^{op}Hrop contains the explicit runtime and environmental premises (such as scheduler fairness and external response guarantees) required for those arguments. Monotonicity of finite trace languages does not transfer these operational properties; they are conjoined at the acceptance boundary.

3.7 Subgraph extraction, root-covering cuts, and frontier soundness

Section titled “3.7 Subgraph extraction, root-covering cuts, and frontier soundness”

For $S \subseteq V$S ⊆ V, let $\mathrm{Anc}(S)$Anc(S) and $\mathrm{Desc}(S)$Desc(S) be reflexive ancestor and descendant closures. The interval from uu to ww is $\mathrm{Desc}(\{u\}) \cap \mathrm{Anc}(\{w\})$Desc({u}) ∩ Anc({w}). A **root-covering cut** FF is an antichain intersecting every root-to-leaf path. These are semantic inspection boundaries, not graph-layout levels; different branches may meet FF at different depths.

For $v \in \mathrm{Anc}(F)$v ∈ Anc(F) define

$$U_F(v) = \begin{cases} \mathrm{tr}(M_v), & v \in F, \\ K_v((T_{v,c}(U_F(c)))_c), & v \notin F. \end{cases}$$

UF(v) = {tr(Mv),Kv((Tv,c(UF(c)))c),v ∈ F,v ∉/ F.

This recursion is well typed. If vv is above the cut and one child had no descendant in FF, extending that child path to a leaf would bypass FF; an earlier cut ancestor would contradict the antichain property. Thus all required children are in the upper region. Shared state-free nodes are evaluated once with their edge-indexed transformers.

Corollary 1 (Frontier Soundness). Suppose frontier realization $\mathrm{tr}(N_f) \subseteq \mathrm{tr}(M_f)$tr(Nf) ⊆ tr(Mf) holds for every $f \in F$f ∈ F. Suppose the upper region has the edge, decomposition, guarded wiring, actual-witness admission, and scope premises of Theorem 1, and roots satisfy their active intents. Under the full closure of these premises,

$\mathrm{tr}(N_r) \subseteq U_F(r) \subseteq \mathrm{tr}(M_r) \subseteq \Phi(r)$.tr(Nr) ⊆ UF(r) ⊆ tr(Mr) ⊆ Φ(r).

Proof. Simultaneously induct upward on $\mathrm{tr}(N_v) \subseteq U_F(v) \subseteq \mathrm{tr}(M_v)$tr(Nv) ⊆ UF(v) ⊆ tr(Mv). The frontier case is assumed. Above it, the right inclusion follows from TT-monotonicity, edge refinement, and decomposition. The left inclusion follows from guarded wiring, whose slot guards follow from the same inductive bounds, and monotonicity into $U_F(v)$UF(v). Root coverage makes the induction cover every root. □□

The root-covering condition ensures that every execution path to a leaf is intercepted by the cut; omitting any branch would discard component behaviors required for parent soundness. Because abstract frontier models provide safe over-approximations of executable behaviors, the theorem yields behavioral containment rather than trace equivalence.

Root coverage guarantees the sound upward transfer of finite-safety trace containment. It does not imply that readiness, unresolved duties, external wait statuses, or human comprehension can be evaluated solely from the cut. Operational progress and interactive inspection require evidence tied directly to the relevant execution witnesses and interfaces.

3.8 Protected boundary inventory

Section titled “3.8 Protected boundary inventory”

Define a tagged disjoint union

$\mathrm{Prot}(G) = \mathrm{Prot}_{\mathrm{intent}}(G) \uplus \mathrm{Prot}_{\mathrm{slot}}(G)$.Prot(G) = Protintent(G) ⊎ Protslot(G).

The intent part contains each active `root` or `protected` record, including identity, owner, authority, predicate, and scope. The slot part contains every parent–owned record

$((v, c), B_{v,c}, \lambda_{v,c}, A_{v,c}, \mathrm{scope}_{v,c})$.((v, c), Bv,c, λv,c, Av,c, scopev,c).

Scope encompasses the formal observation profile, action mapping, declared duty and wait policies (E_d^{∞}Ed∞), and environmental context. The protected inventory is a derived projection of accepted intent and slot records, not a duplicate authoring artifact.

Across development steps, governing meaning is preserved by evaluating the evolution condition against the prior protected inventory:

$$\forall p \in \mathrm{Prot}(G_t).\quad \mathrm{Preserved}(p, G_{t+1}) \;\vee\; \exists a.\, \mathrm{AuthorizedDisposition}(a, p, G_{t+1}). \qquad (\mathrm{Protect})$$
∀p ∈ Prot(Gt). Preserved(p, Gt+1) ∨ ∃a. AuthorizedDisposition(a, p, Gt+1).(Protect)

Here $\mathrm{Preserved}(p, G_{t+1})$Preserved(p, Gt+1) asserts that the same protected subject and governing meaning remain in force under the same applicable scope; for an active protected intent, this includes continued activation in the support graph. An authorized disposition identifies the prior protected record and records its accepted replacement or intentional retirement under entitled authority. Changes to supporting intents that cause a protected intent to lose activation require an explicit disposition, precisely as direct deletion does.

Authorizations may be granted via standing architectural delegation. Routine implementation adjustments and local refactorings within an authorized grant do not require interactive human re–approval, cryptographic tokens, or third–party signing services. However, modifying a slot contract (such as enlarging a language from $\{\epsilon, ok\}$ϵ, ok to $\{\epsilon, ok, bad\}$ϵ, ok, bad), altering a response duty, or strengthening environmental premises alters protected interface meaning and requires formal architectural re–authorization.

3.9 Human governance and inspectable projections

Section titled “3.9 Human governance and inspectable projections”

Human governance in agent–authored systems operates in two complementary directions: upstream intent authoring and downstream interactive assessment.

During authoring, stakeholders and automated agents must be able to express, inspect, and revise intent from relevant boundary responsibilities, interaction protocols, and operating conditions, without being forced to inspect or reconstruct unrelated implementation code, internal component state machines, or low–level proof scripts. Encapsulated slot contracts and single–owner access boundaries establish the modular units for this authoring reduction.

During assessment, human stakeholders must be able to visually inspect system status, active commitments, component boundaries, recent modifications, evidence strength, unresolved response duties, and external wait prerequisites. Decision–critical context must be exposed directly, while detailed proof transcripts, leaf handler definitions, and full execution traces remain accessible on demand.

A root–covering frontier (§3.7) provides a sound mathematical abstraction for finite–safety inspection, allowing a reviewer to evaluate system safety against high–level component contracts rather than leaf implementations. However, a frontier cut does not by itself establish reduced human cognitive burden:

- **Decision–dependent context:** The context required to make a sound governance decision depends on the question being asked. Local questions require narrow views; cross–cutting architectural changes may require global system context. An inspection projection must not create a false impression of local safety by displaying an incomplete, non–covering cut.
- **Inspectable failure:** When a failure occurs or a verification check fails, the diagnostic view must identify the affected duty, the reporting boundary, the violated condition, and any remaining uncertainty. It must not manufacture an unverified root–cause attribution.
- **Faithful wait views:** A genuine external wait must be displayed accurately as pending at its identified prerequisite ($W_{d,e}$Wd,e). It must never be presented as a successful completion or an unexplained hang.
- **Assurance status:** Any rendered assertion must faithfully reflect the evidential status of the underlying record, distinguishing proved theorems, bounded tests, empirical observations, and refutable hypotheses.

The formal calculus defines these governance requirements and inspection boundaries; it does not mandate a specific UI framework, layout algorithm, or visualization widget, and claims no measured empirical reduction in human effort.

4. Verification Obligations and Restricted Leaf Realization

Section titled “4. Verification Obligations and Restricted Leaf Realization”

4.1 Satisfaction

Section titled “4.1 Satisfaction”

Satisfaction verifies $\mathrm{tr}(M_v) \subseteq [\![j]\!]$ tr(Mv) ⊆ [[j]] for each active owned intent. Depending on state complexity, this obligation may be discharged through exhaustive model checking, inductive invariant verification, or sound abstract interpretation. Where finite test suites are employed, universal inclusion requires an accompanying coverage proof demonstrating that all reachable behaviors have been explored.

4.2 Decomposition

Section titled “4.2 Decomposition”

Decomposition establishes $K_v((L_{v,c})_c) \subseteq \mathrm{tr}(M_v)$Kv((Lv,c)c) ⊆ tr(Mv) strictly against abstract slot contracts rather than concrete child implementations. Consequently, the resulting proof remains valid for any replacement child components that satisfy the same boundary contracts. While the mathematical trajectory definition guarantees monotonicity, the operational fidelity of the composition is established through the guarded wiring obligation.

4.3 Child refinement and deterministic contract monitors

Section titled “4.3 Child refinement and deterministic contract monitors”

A Level–3 monitor is

$$B = (Q, q_0, \Sigma^{\mathrm{in}} \uplus \Sigma^{\mathrm{out}}, \beta), \qquad \beta \colon Q \times \Sigma^{\mathrm{vis}} \rightharpoonup Q.$$

B = (Q, q0, Σin ⊎ Σout, β), β : Q × Σvis ⇀ Q.

It has one initial state, no hidden transitions, and at most one successor for each state and label. Every reachable state accepts a finite prefix; missing transitions forbid events. Distinct labels such as `ok` and `fail` may both be enabled. Determinism does not imply finite state. Every prefix–closed language LL containing εϵ has a canonical such monitor: states are words in LL, the initial state is εϵ, and $w \xrightarrow{a} wa$ w a

---

wa exactly when $wa \in L$wa ∈ L.

Extend λλ to internal source labels by erasure. An initialized forward simulation $R \subseteq S_N \times Q$R ⊆ SN × Q relates every source initial state to q_0q0. For every related pair and every source step $s \xrightarrow{a} s'$s a

---

s′, it requires $(s', q) \in R$(s′, q) ∈ R when $\lambda(a) = \varepsilon$λ(a) = ϵ, and otherwise requires $\beta(q, \lambda(a)) = q'$β(q, λ(a)) = q′ and $(s', q') \in R$(s′, q′) ∈ R.

Proposition (Simulation Completeness for Deterministic Monitors).

$$\lambda^*(\mathrm{tr}(N)) \subseteq \mathrm{tr}(B) \iff \text{an initialized forward simulation } N \to B \text{ exists.}$$

λ∗(tr(N)) ⊆ tr(B) ⟺ an initialized forward simulation N → B exists.

Proof. A simulation matches each finite run by induction. Conversely, relate ss to every monitor state reached by the mapped trace of a source execution ending in ss. Inclusion admits that trace. Extend its witnessing execution by any source step; determinism makes the monitor state after its old prefix the very state already related to ss, so the new label matches or erases as required. □□

This proposition establishes simulation completeness strictly for finite visible-trace inclusion. It does not establish preservation of operational readiness, non-silent response, or divergence freedom. For scoped edge refinement, the simulation equivalence applies directly to the restricted witness $\mathrm{Restrict}_{A_{v,c}}(N)$RestrictAv,c(N), while relative input reception and readiness are independently verified on the operational witness (relative reception may share the finite product traversal while remaining a distinct predicate).

The deterministic-target premise is essential. Consider the nondeterministic target with edges $0 \xrightarrow{a} 1 \xrightarrow{b} 20$ a

---

1 b

---

2 and $0 \xrightarrow{a} 2 \xrightarrow{c} 30$ a

---

2 c

---

3. Its language is $\{\epsilon, a, ab, ac, abc\}$$\{$ϵ, a, ab, ac, abc$\}$: the shared state 22 also permits cc after abab. A source $s_0 \xrightarrow{a} s_1$s0 a

---

s1, $s_1 \xrightarrow{b} s_2$s1 b

---

s2, $s_1 \xrightarrow{c} s_3$s1 c

---

s3, $s_2 \xrightarrow{c} s_3$s2 c

---

s3 has exactly that language. No simulation exists: after aa, target state 11 cannot match the source's cc, and state 22 cannot match its bb. Equality of finite trace languages therefore does not suffice for this nondeterministic target.

4.4 Leaf realization and Restricted Leaf Normal Form

Section titled "4.4 Leaf realization and Restricted Leaf Normal Form"

An RLNF leaf is

$D_\ell = (S_\ell, s_{0,\ell}, I_\ell, O_\ell, \mathrm{en}_\ell, \delta_\ell)$, Dℓ = (Sℓ, s0,ℓ, Iℓ, Oℓ, enℓ, δℓ),

where $s_{0,\ell} \in S_\ell$s0,ℓ ∈ Sℓ, $\mathrm{en}_\ell : S_\ell \to P(I_\ell)$enℓ : Sℓ → P(Iℓ), and

$\delta_\ell : \{(s,i) \mid i \in \mathrm{en}_\ell(s)\} \longrightarrow S_\ell \times \mathrm{List}(O_\ell)$δℓ : {(s, i) | i ∈ enℓ(s)} ⟶ Sℓ × List(Oℓ)

is pure, single-valued, total on its domain, and terminating. All persistent state influencing handler execution is encapsulated within S_ℓSℓ or supplied via explicit inputs. Total functional programming primitives, explicit structural recursion, or verified runtime substrates discharge handler termination. Handler termination guarantees that each individual invocation finishes; it does not by itself prove termination of an entire chain of interacting handlers across a connector, nor does it prevent connector-level communication deadlocks.

The microstep elaboration has configurations $S_\ell \times \mathrm{List}(O_\ell)$Sℓ × List(Oℓ), initial configuration $(s_{0,\ell}, [])$(s0,ℓ, []), and exactly two rules:

$$\frac{i \in \mathrm{en}_\ell(s) \quad \delta_\ell(s,i) = (s', w)}{(s, []) \xrightarrow{\mathrm{in}(i)} (s', w)} \qquad \frac{}{(s, o::w) \xrightarrow{\mathrm{out}(o)} (s, w)}.$$

(s, []) in(i)

No input dispatch occurs while outputs remain in the pending list. The explicit state may already hold the next handler state while the output list records an intermediate observable control point. Pending output lists constitute operational state: a configuration `(Done, [ack(i)])` has not yet discharged its root response duty until the output event is emitted. An empty output list is valid whenever the

contract owes no immediate response; it does not indicate error, but neither does returning `[]` excuse an unfulfilled response obligation. Empty lists permit the next enabled input immediately.

For any finite handler chain $s_0 \overset{i_1/w_1}{\Rightarrow} s_1 \cdots \overset{i_n/w_n}{\Rightarrow} s_n$s0 i1/w1

---

s1 ⋯ in/wn

---

sn, form the concatenated packets $i_1 w_1 \cdots i_n w_n$i1w1 ⋯ inwn, with input/output tags understood. Let $\mathrm{Packets}(D)$Packets(D) contain all such words, including ϵϵ.

Lemma (Packet Decomposition).

$\mathrm{tr}(\mathrm{elab}(D)) = \mathrm{pref}(\mathrm{Packets}(D))$.tr(elab(D)) = pref(Packets(D)).

Proof. A finite elaborated execution consists of completed packets and possibly a final packet whose output list is only partly drained. Complete that finite list to obtain a packet word extending the trace. Conversely, execute each packet's input step and output drains; any prefix of that execution is a valid finite execution. The zero-packet and empty-output cases follow from the same rules. □□

Substrate operations are externalized:

$$I_\ell = I_{\mathrm{env}} \uplus \mathrm{Resp}_\ell, \qquad O_\ell = \mathrm{Emit}_\ell \uplus \mathrm{Cmd}_\ell.$$

Iℓ = Ienv ⊎ Respℓ, Oℓ = Emitℓ ⊎ Cmdℓ.

A shell Sh_ℓShℓ is a formal transition system that receives command outputs and emits corresponding response inputs. The connector specifies synchronization mechanisms, buffering boundaries, response correlations, and outcome domains. The formal shell model captures observable interface interactions, with the physical hardware and substrate fidelity explicitly designated as environmental premises in the trusted base.

The mathematical realization obligation is

$$\mathrm{tr}(\mathrm{elab}(D_\ell)) \restriction_{A_\ell} \subseteq \mathrm{tr}(M_\ell).$$

tr(elab(Dℓ)) ↾Aℓ ⊆ tr(Mℓ).

When using host-language implementations P_ℓPℓ, an explicit translation premise Trans_ℓTransℓ formally establishes that the compiled or interpreted runtime witness refines the RLNF microstep elaboration under identical observations, input scopes, and substrate bindings. Composing this translation refinement with the tabular realization proof completely discharges ImplImpl for finite visible-trace safety.

Operational properties—relative reception, readiness, and non-silent response—require evidence about the actual runtime execution witness or an explicit correspondence preserving those properties. Trace containment alone cannot establish them. When the mathematical elaboration is itself the executable artifact, identity supplies this correspondence. Running the elaboration through a host runtime or interpreter introduces a runtime boundary; that boundary must be justified by its execution semantics rather than assumed identical by naming the source table. The single ImplImpl obligation encompasses this realization chain, maintaining distinguishable evidence scopes without requiring artificial API splits.

Here inputs denote dispatch. An asynchronous arrival buffer, if used, has separate enqueue and dispatch transitions; dispatch removes its head only when the leaf is ready and that input is enabled. Pending output drains never remove buffered inputs. Such a buffer is an owned component or substrate primitive requiring its own conformance evidence. The synchronous connector in §8 needs no mailbox. In either case, an empty output list in `AwaitFsync` enables only its declared completion inputs, not a new ordinary request.

4.4.1 Verifying readiness and complete-execution duties

Section titled "4.4.1 Verifying readiness and complete-execution duties"

Verifying the operational obligations of §2.5 requires techniques tailored to execution witnesses rather than language containment:

- **Readiness (Ready):** For finite-state systems, readiness is verified by reachable-configuration exploration. At each reachable configuration (s, h), the checker confirms that either an external wait $W_{d,e}$ holds or at least one owned continuation step $s \xrightarrow{\text{owned}(d)} s'$ is enabled. For synchronous connectors, the check evaluates composite states to confirm that the recipient is receptive to the proposed step.
- **Non-silent response (NS):** Verifying NS requires analyzing complete maximal executions in $\text{CExec}_v(N_v, \omega)$. A reachable terminal or quiescent configuration where duty d is outstanding without an active genuine external wait constitutes an immediate counterexample. For infinite executions, finite-state cycle detection over the synchronized product evaluates whether any reachable cycle contains outstanding duties without emitting a permitted outcome or persistently remaining in a permitted wait ($e \in E_d^{\infty}$). Any such cycle must be shown unreachable, or shown to be excluded by an explicit, warranted scheduling or fairness premise.
- **Deductive ranking:** For systems with infinite state spaces or complex control loops, developers may supply a well-founded ranking function or variant that strictly decreases across owned continuation steps toward outcome emission or wait entry.

The calculus does not mandate syntactically acyclic code, automatic ranking synthesis, universal strong fairness, or artificial watchdog timers. A loop with cyclic control syntax is fully admissible provided its execution terminates or satisfies the required response condition. The evidence record simply documents the verification method, its completeness, its assumptions, and the runtime conditions under which progress is assured.

4.5 Evidence discipline

Section titled “4.5 Evidence discipline”

An evidence record explicitly documents the obligation, target claim, verification method, execution outcome, underlying assumptions, architectural scope, and verified artifact. Different verification approaches—deductive model proofs, exhaustive finite exploration, bounded analysis, runtime conformance observations, and empirical human-task evaluations—provide distinct formal assurances and must remain strictly separated. Missing or incomplete evidence leaves an obligation unresolved; while an empty result may be valid for a generic query, the absence of a required subject cannot be reported as coverage of that subject. Following the foundational principles of the Evidential Tool Bus (Rushby, 2005; Cruanes et al., 2013, 2014) and Assurance 2.0 (Bloomfield, Netkachova, and Rushby, 2024), verification results are maintained as structured, typed arguments, ensuring that tool evidence directly matches the strength and assumptions of its claimed obligation.

4.6 Three-gate structure

Section titled “4.6 Three-gate structure”

The evidence gate evaluates three orthogonal dimensions of acceptance: **behavioral validity**, confirming mathematical trace inclusions and committed operational/temporal duties; **authority preservation**, verifying that protected intents and slot contracts conform to architectural authorizations; and **evidence applicability**, confirming that proof artifacts, environmental scopes, and candidate binaries match the target obligation precisely.

4.7 Consolidated trusted base

Section titled “4.7 Consolidated trusted base”

The following inventory collects the premises that connect the mathematical argument to an implementation. Items proved in a particular deployment cease to be assumptions there, but their boundary obligations do not disappear.

Boundary	Required evidence or explicit premise	What is not inferred
Intent and context interpretation	Accepted predicates, authority, scopes, feasible environment interpretation	That a formalized requirement matches every unstated user expectation
Mathematical proof / checker	Sound inference and applicable checker semantics or checked certificates	Universal validity from a bare process exit code

Boundary	Required evidence or explicit premise	What is not inferred
RLNF handlers	Totality, purity, explicit-state closure, enabled-input definition	Termination from syntax alone or state-freedom from an empty object
Source-to-RLNF translation	Trans_ℓTransℓ specifying exact code, runtime, and preserved properties	Correctness of arbitrary host-language code or runtime progress from trace safety
Runtime and connector	AdmAdm, ownership, relative reception, guarded Wire Wire, scope erasure, and scheduling/execution semantics for progress	Legal delivery or progress from local trace inclusion alone
Physical substrate	Command/response meaning, failure model, durability hypothesis, and conditional external-response premises (including H_{write}Hwrite)	Physical persistence or eventual response from symbolic labels or shell graphs alone
Evolution and evidence storage	Operand identity or proved transport; coherent candidate publication and preserved audit records	Semantic equivalence from labels, paths, or cryptographic hashes alone
Human reading and inspection	Faithful decision-relevant projections and appropriately scoped evaluation tasks	Cognitive context reduction or governance correctness from graph proofs or visualizations alone

5. Decomposition, Modularity, and Interaction

Section titled “5. Decomposition, Modularity, and Interaction”

5.1 Contracts are relational

Section titled “5.1 Contracts are relational”

Proposition 1 (Coordinate Projections Erase Correlation). A relation need not equal the product of its coordinate projections.

Proof. For $R = \{(0,0),(1,1)\}$R = {(0, 0), (1, 1)}, both projections are $\{0,1\}${0, 1}. Their product also contains $(0,1)$(0, 1) and $(1,0)$(1, 0), which are not in RR. □□

Accordingly, independent child predicates do not establish an interaction property such as matching transaction identifiers or coherent shared-state updates. The owning connector’s joint trajectories retain that correlation. Decomposition verifies the property where the interaction is owned.

5.2 Sibling independence and safety substitution

Section titled “5.2 Sibling independence and safety substitution”

Theorem 2 (Safety Substitution). Fix a parent’s context, admission domain, contracts, abstractions, input scopes, and connector. If replacement witnesses are admissible and satisfy every fixed slot, the same guarded wiring and decomposition proofs establish their composite inclusion in $\mathrm{tr}(M_\upsilon)$tr(Mv). An unchanged sibling needs no new local refinement proof solely because another child was replaced.

Proof. Instantiate the existing universal wiring certificate with the replacement tuple. Slot conformance gives its guard. Monotonicity and the fixed decomposition yield the inclusion, exactly as in Theorem 1. □□

Theorem 2 establishes formal proof modularity for finite visible-trace safety: verifying a localized component replacement preserves parent safety whenever external slot contracts are respected.

This locality principle applies to other behavioral properties only when accompanied by an applicable composition or transport argument. In particular, Theorem 2 does not establish compositional progress or non-silent response. Mutually dependent progress obligations can form circular dependencies: component A waits for an event from B, while B waits for an event from A. Both local conditional progress obligations may hold vacuously while the composite system deadlocks. A parent response commitment must verify composite execution explicitly to ensure that circular waiting is absent.

5.3 Executable closure

Section titled “5.3 Executable closure”

Every executable behavior contributing to a root system is explicitly owned by an RLNF leaf, a declared connector transition, or a declared substrate primitive. Architectural routing skeletons synchronize and map events, while persistent queues, transaction managers, and shared scheduling mechanisms are represented as explicit owned components or declared runtime/substrate state, with the applicable proof or hypothesis retained in the dependency closure.

5.4 Single-owner shared services via parent-connector serialization

Section titled “5.4 Single-owner shared services via parent-connector serialization”

A parent component may own a stateful service and mediate requests originating from multiple client branches. A serializing connector coordinates client access by granting service access to at most one outstanding operation at a time, recording client identities, and delivering responses back to the originating client. The shared state is owned explicitly by a coordinator or verified runtime primitive.

The serializing connector chosen here orders service invocations. The shared-counter example illustrates its aggregate contract; shared mutable state alone does not impose this particular serialization strategy. Interleaved or pipelined access can be supported by adopting alternative connector semantics, such as linearizable concurrent data structures (Herlihy and Wing, 1990; Filipovic´et al., 2010), provided the resulting joint trajectories are verified against the parent contract.

5.5 Verified and substrate libraries

Section titled “5.5 Verified and substrate libraries”

Verified third-party libraries integrate into the development DAG through declared interfaces, applicable refinement evidence, and clear runtime ownership bindings. Components that fall outside RLNF are modeled as declared substrate boundaries supported by an explicit hypothesis or a separately established translation or refinement result. In accordance with compositional design, specifications adopt the minimal contract strength necessary to establish parent requirements; any relied-on progress promise requires its own applicable argument.

5.6 Admissible-input databases

Section titled “5.6 Admissible-input databases”

General storage interfaces frequently support rich query protocols, whereas an application may rely upon a disciplined transaction sequence. By formalizing this usage pattern as an input scope AA and proving that the owning connector delivers only admissible sequences, component refinement is established on the relevant operating domain, provided that required operating uses are not excluded from AA merely to pass a check. Durability guarantees, transaction isolation levels, and failure models are formalized as explicit boundary premises, ensuring that transactional semantics remain verified relative to those premises.

6. The Development Protocol

Section titled “6. The Development Protocol”

6.1 Five development gates

Section titled “6.1 Five development gates”

The development protocol organizes verification into five gates, distinct from the three evidence dimensions in §4.6.

Gate	Required result
G1: Declare	Fix accepted intents, scopes, authority, response duties, permitted failure conditions, external wait policies (E_d^{∞}Ed∞), and the contract-inclusive $\mathrm{Prot}(G)$ Prot(G). Separate open questions and hypotheses from commitments.
G2: Model	Establish typed machines, contexts, input protocols, ownership, and satisfaction obligations without deactivating failed intents.
G3: Decompose	Fix parent slots and connector semantics; prove decomposition and guarded wiring for the stated witness domain.

Gate	Required result
G4: Realize	Prove child refinements, RLNF realizations, operational readiness, translation premises where needed, and actual-witness admission.
G5: Integrate	Check the full dependency closure, all applicable evidence, operational non-silent response, protected dispositions, and the integrated candidate before publication.

These gates structure development responsibilities and evaluation criteria. They do not mandate a serialized, multi-pass tool pipeline; a unified analysis engine may check multiple gates during a single traversal. Publication requires that all five gates pass; an unresolved operational duty cannot be published merely because finite-safety checks succeeded.

6.2 Authoring cycle and discriminating regressions

Section titled “6.2 Authoring cycle and discriminating regressions”

When introducing a modification, developers identify the target requirement and its owning architectural boundary. Target specifications and discriminating test cases are established prior to implementation. Discriminating cases must explicitly cover critical failure modes: silent duty loss, enabled-but-untaken continuations, genuine external waiting, resolved-but-undrained pending outputs, and composite mutual waiting. Expected rejections are determined by the formal specification, never copied from candidate tool output. Complete verification records, counterexamples, and regression suites are retained across iterations to support rigorous evaluation.

6.3 Authority and repair

Section titled “6.3 Authority and repair”

An implementation author may modify internal component behavior while preserving incoming slot contracts. A slot revision, an intent revision, and an environmental assumption change are distinct architectural actions requiring separate authorization. Editing a test to match an implementation does not establish the original requirement.

When an implementation defect is identified, it is repaired directly within the existing component boundary. Governing revisions may proceed under an applicable standing delegation; notification of proof success does not by itself constitute authorization. Routine engineering decisions are resolved within authorized grants. Accepted informal or unresolved intent remains governing: if a material conflict or ambiguity arises in accepted system requirements, only the affected action is paused and escalated for an explicit architectural decision, while independent authorized work continues; until every accepted commitment governing a target scope is formalized into $\mathrm{Prot}(G)$Prot(G) and discharged (or explicitly disposed by an authorized decision), publication of that scope remains `Unresolved`. Once a protected revision is authorized, all resulting validity obligations still apply.

7. Versioned Evolution and Incremental Frame Cutoff

Section titled “7. Versioned Evolution and Incremental Frame Cutoff”

7.1 Versioned developments

Section titled “7.1 Versioned developments”

A version sequence $(G_t)_t$(Gt)t retains prior records and evidence. A candidate G_{t+1}Gt+1 is distinct from a verified published version. Let $\mathrm{Req}(G)$Req(G) be its set of typed scoped obligations, including all finite-safety inclusions, relative reception, operational readiness, and contract-relative non-silent response duties. Publication checks the protected inventory via condition $(\mathrm{Protect})$(Protect) on prior protected subjects (§3.8), alongside all structural, admission, context, and integration premises; merely discharging changed formulas is insufficient.

An authorized disposition records the original protected subject, its accepted updated specification or retirement, the governing authority, and the applicable scope. Across each published version, logical cells resolve to coherent definitions, with distinct stateful runtime instances maintaining unique runtime identities.

7.2 Contextual projection and staleness

Section titled “7.2 Contextual projection and staleness”

When an accepted intent motivates a stronger child contract or an alternative decomposition, the proposed structure is validated by re-establishing parent decomposition and context entailments. Modifications propagate along semantic dependency paths: downstream obligations remain pending until prior evidence is demonstrated to be applicable (by exact operand identity or a justified transport argument) or fresh proofs are supplied. A modification requires examining affected applicability rather than automatically reproving unaffected subgraphs.

7.3 Exact obligation operands and the frame theorem

Section titled “7.3 Exact obligation operands and the frame theorem”

Let σ_qσq contain the typed interfaces, visible/internal interpretation, profile, accepted context with transitive hypothesis meanings, duty interpretations, wait policies (E_a^{∞}Ed∞), inspectability mappings, and runtime scheduling premises used by obligation qq. With ordered slot identities S_vSv, the five operand records are:

Family	Exact typed operands
$\mathrm{Sat}(v, j)$Sat(v, j)	$(X_v, M_v, [\![j]\!], \sigma_q)$(Xv, Mv, ⟦j⟧, σq)
$\mathrm{Wire}(v)$Wire(v)	$(S_v, (X_c, X_{v,c})_c, X_v, (\mathrm{Adm}_c)_c, C_v, W_v, K_v, (B_{v,c}, \lambda_{v,c}, A_{v,c})_c, \mathrm{Delivery}_v, R_v, \sigma_q)$(Sv, (Xc, Xv,c)c, Xv, (Admc)c, Cv, Wv, Kv, (Bv,c, λv,c, Av,c)c, Deliveryv, Rv, σq)
$\mathrm{Dec}(v)$Dec(v)	$(S_v, K_v, (L_{v,c})_c, M_v, \sigma_q)$(Sv, Kv, (Lv,c)c, Mv, σq)
$\mathrm{Ref}(v, c)$Ref(v, c)	$(X_c, X_{v,c}, M_c, B_{v,c}, \lambda_{v,c}, A_{v,c}, \sigma_q)$(Xc, Xv,c, Mc, Bv,c, λv,c, Av,c, σq)
$\mathrm{Impl}(\ell)$Impl(ℓ)	$(X_\ell, P_\ell, D_\ell, \mathrm{elab}, A_\ell, M_\ell, R_\ell, \mathrm{Trans}_\ell, \sigma_q)$(Xℓ, Pℓ, Dℓ, elab, Aℓ, Mℓ, Rℓ, Transℓ, σq)

The configuration RR formalizes runtime instance allocation, substrate shell and buffer models, and scoped witness constructors. In direct formalizations where the executable implementation is the mathematical model D_ℓDℓ, $P_\ell = D_\ell$Pℓ = Dℓ and Trans_ℓTransℓ is witnessed by identity. For coarse slots, replace monitor $B_{v,c}$Bv,c by the declared slot language $L_{v,c}$Lv,c. Operational evidence for readiness and non-silent response depends on the execution witness, duty definitions, wait policies, and scheduling premises encoded in σ_qσq.

Guarded WireWire depends on BB when its delivery proof uses slot conformance. Changing a public slot can therefore change WireWire as well as DecDec and RefRef. Replacing a child witness does not change the universal quantifier’s domain merely because that witness has different internal states; changing the admission domain does.

Exact operand identity can be conservatively recognized by comparing complete normalized typed records under their declared interpretations. Version identifiers and cryptographic hashes may index these records, but hashes establish neither semantic equivalence nor discovery of omitted dependencies; a modified claim requires fresh verification or a justified transport argument.

We partition required obligations in G_{t+1}Gt+1 by evidence applicability:

$$\begin{aligned} \mathrm{Reuse}_{t \to t+1} &= \{q \in \mathrm{Req}(G_{t+1}) \mid \exists e \in E_t.\ \mathrm{Applicable}(e, q, G_{t+1})\}, \\ \mathrm{Check}_{t \to t+1} &= \mathrm{Req}(G_{t+1}) \setminus \mathrm{Reuse}_{t \to t+1}. \end{aligned} \qquad \text{(Frame partition)}$$

Reuset→t+1Checkt→t+1 = {q ∈ Req(Gt+1) | ∃e ∈ Et. Applicable(e, q, Gt+1)}, = Req(Gt+1) ∖ Reuset→t+1 .(Frame partition)

Here $\mathrm{Applicable}(e, q, G_{t+1})$Applicable(e, q, Gt+1) asserts that prior valid evidence artifact ee establishes typed claim qq under the new candidate’s environment, either through identical typed operands and uninvalidated context or via an explicit, sound proof-transport result. Stable obligation names or display IDs alone do not establish applicability: an obligation whose operand changed under the same name without an applicable proof-transport result belongs to $\mathrm{Check}_{t \to t+1}$Checkt→t+1.

Theorem 3 (Incremental Frame). Suppose every claim in $\mathrm{Reuse}_{t \to t+1}$Reuset→t+1 has applicable evidence carried forward from G_tGt, every claim in $\mathrm{Check}_{t \to t+1}$Checkt→t+1 is discharged with new valid evidence, and candidate G_{t+1}Gt+1 satisfies all structural, context, admission, integration, and protected-preservation conditions. Then all required claims of the selected profile in G_{t+1}Gt+1 have valid applicable evidence. Its safety subfamily satisfies Conditions 1–6, so Theorem 1 holds under the new dependency context; any separately required operational claims hold by their respective evidence.

Proof. By definition, $\mathrm{Reuse}_{t \to t+1}$Reuset→t+1 and $\mathrm{Check}_{t \to t+1}$Checkt→t+1 partition $\mathrm{Req}(G_{t+1})$Req(Gt+1). Their associated evidence therefore covers every required claim of the new candidate, including obligations with modified operands under preserved names. The remaining candidate premises satisfy Conditions 1–6 and condition $(\mathrm{Protect})$(Protect). Applying Theorem 1 to the safety subfamily yields root safety; operational claims hold by their verified conjuncts. □□

7.4 Corollary 2: Interface cutoff

Section titled “7.4 Corollary 2: Interface cutoff”

Corollary 2 (Interface Cutoff). Replace a region while fixing all external boundary operands: slot contracts, maps, input scopes, parent model and intents, connector and admission domain, runtime interpretation, and contexts. Once the replacement’s local and incoming refinement obligations are established, the parent WireWire and DecDec proofs are reusable, as are other operand-identical obligations with applicable evidence.

Proof. Universal wiring does not contain a particular child’s implementation as an operand. The fixed parent decomposition contains slot languages rather than child models. Their typed propositions are unchanged; apply Theorem 3. □□

A leaf-code change with fixed M_ℓMℓ changes realization but not incoming model refinement. A child-model change can change incoming refinement without changing the fixed parent slot. A split or merge changes the direct parent’s slot signature and wiring, so cutoff occurs above the first preserved boundary, not at the modified connector itself. Changing a slot guard, input scope, substrate meaning, or runtime interpretation is not an unchanged-boundary case.

Reusing operational readiness or progress arguments similarly requires verifying that underlying execution dependencies and scheduling premises remain preserved or justified by an applicable transport argument. An unchanged safety slot alone is insufficient; conversely, a localized, progress-preserving leaf modification does not invalidate unrelated sibling components. Universal parent proof reuse is also distinct from caching implementation-sensitive diagnostics, which depend on concrete child realizations rather than abstract slot contracts alone.

This result provides an obligation-level instance of early cutoff: re-verification of unaffected subgraphs terminates as soon as external interface operands are confirmed unchanged, adapting the dependency-tracking principles of modern build systems (Mokhov, Mitchell, and Peyton Jones, 2018) to formal verification obligations.

7.5 Typed evolution and four-way failure diagnosis

Section titled “7.5 Typed evolution and four-way failure diagnosis”

Define the tagged outcome type

$$\begin{aligned}\mathrm{EvolutionResult} = \quad & \mathrm{Verified}(\mathrm{Development}, \mathrm{Evidence}, \mathrm{AuthorizedDispositions}) \\ & \mid \mathrm{Conflict}(\mathrm{Development}, \mathrm{ConflictWitness}) \\ & \mid \mathrm{Unresolved}(\mathrm{Development}, P_{\mathrm{fin}}(\mathrm{Obligation})).\end{aligned}$$

EvolutionResult = Verified(Development, Evidence, AuthorizedDispositions) | Conflict(Development, ConflictWit

An evolution procedure has the partial-function type

$$\mathrm{evolve}: \mathrm{Development} \times \mathrm{AuthorizedDelta} \rightharpoonup \mathrm{EvolutionResult}.$$

evolve : Development × AuthorizedDelta ⇀ EvolutionResult.

The payload types represent the possible outcomes of development evolution: `Verified` denotes a candidate that satisfies all publication and verification gates across the entire selected profile (including committed operational duties, not safety alone); `Conflict` documents a demonstrated incompatibility among accepted requirements or environmental premises; and `Unresolved` captures open proof obligations and pending implementation adjustments requiring further automated or human intervention. A pending external request at runtime is an operational execution status, not one of these verification-outcome constructors.

After applying an authorized delta, recompute active intents, exact obligations, dependency closure, and protected dispositions. Check premise compatibility and evidence applicability. A failed attempt is classified into four evidentially distinct cases:

1. **Concrete violation:** a legal joint executable run violates the accepted predicate. Repair the implementation or owning integration boundary.
2. **Abstract counterexample:** a model or contract trajectory violates its target, but runtime feasibility is not established. Refine the offending abstraction or produce a concrete witness before calling it a runtime bug.
3. **Failed proof witness:** a proposed invariant or simulation does not close. Repair the proof witness or investigate further; this does not itself refute inclusion.
4. **Unknown:** timeout, unsupported reasoning, or missing evidence. Preserve the open obligation without classifying it as true, false, genuine, or spurious.

This taxonomy synthesizes counterexample-guided abstraction refinement (Clarke et al., 2003) with the eliminative argumentation of Assurance 2.0 (Bloomfield, Netkachova, and Rushby, 2024), clearly distinguishing concrete runtime bugs, spurious abstraction artifacts, and solver incompleteness.

7.6 Joint trajectory feasibility and repair direction

Section titled “7.6 Joint trajectory feasibility and repair direction”

For an abstract parent counterexample ττ, model-level feasibility is

$$\exists r.\ \mathrm{legal}_{W_v}(r) \wedge \mathrm{obs}_v(r) = \tau \wedge \forall c,\ \mathrm{slice}_c(r) \in T_{v,c}(\mathrm{tr}(M_c)).$$

∃r. legalWv(r) ∧ obsv(r) = τ ∧ ∀c, slicec(r) ∈ Tv,c(tr(Mc)).

Proposition 2 (Joint Trajectory Feasibility). The displayed condition is equivalent to $\tau \in K_v((T_{v,c}(\mathrm{tr}(M_c)))_c)$τ ∈ Kv((Tv,c(tr(Mc)))c). Separate per-child existential witnesses do not imply it.

Proof. The equivalence is the definition of K_vKv. For the second claim, let one shared trajectory bit bb be fixed. One child requires $b = 0$ b = 0 and another requires $b = 1$b = 1. Each has a separate witness; no single trajectory has both values. □□

Establishing the runtime feasibility of an abstract counterexample requires exhibiting a concrete joint execution witness or providing an applicable realization or completeness argument for that trajectory. A forward simulation over-approximation alone is insufficient. Proving a two-way simulation completeness theorem between model and implementation is one sufficient method, but is not mandatory if a direct execution witness is demonstrated.

Identifying the appropriate repair direction depends on whether the defect resides in component implementation or architectural abstraction. For example, if a child component correctly produces $\{\epsilon, ok\}${ϵ, ok} while its abstraction over-approximates behaviors with $\{\epsilon, ok, bad\}${ϵ, ok, bad}, the appropriate repair refines the child abstraction rather than altering the parent slot. Conversely, if a decomposition fails because parent slots permit invalid interleavings, strengthening the slot contracts resolves the issue, provided child components can fulfill the tighter requirements and the contract change is formally authorized. Publish only after all gates pass; otherwise retain the candidate, evidence, and open obligations.

8. End-to-End Worked Example: Durable Acknowledgement

Section titled “8. End-to-End Worked Example: Durable Acknowledgement”

8.1 Requirement and scope

Section titled “8.1 Requirement and scope”

One runtime instance handles at most one request carrying identifier $i \in \mathrm{Id}$i ∈ Id. The visible root requirement is: `ack(i)` is preceded by the matching `commit_ok(i)`; a failure branch emits `commit_fail(i)` and `fail(i)` instead. A named substrate hypothesis H_{dur}Hdur states that a matching successful flush response after the modeled write establishes durability under the declared storage/crash model. It is a forward implication, not an assertion that every durable write must return success.

The environment supplies at most one request. In this modeled scope, file writing produces a successful `write_ok` response, while flush may succeed, fail, or remain pending. Acceptance of `req(i)` opens a root response duty dd, discharged only when root `ack(i)` or permitted root `fail(i)` is emitted. Failure is contractually tied to the modeled unsuccessful flush branch.

In the operational model, the modeled successful write guarantee is formalized as an explicit premise H_{write}Hwrite: following a synchronized `Cmd.write(i)`, the matching response `write_ok(i)` eventually occurs. Consequently, write waiting is strictly transient. In contrast, an unresolved flush operation may remain pending indefinitely as a genuine, inspectable external wait ($E_d^{\infty} = \{\mathrm{flush}(i)\}$Ed∞ = {flush(i)}); the system does not invent a timeout, fabricate an unobserved failure, or assume eventual flush completion. Finite–safety verification does not require H_{write}Hwrite; the scoped progress argument uses it explicitly.

8.2 Root and slot control graphs

Section titled “8.2 Root and slot control graphs”

Every arrow below is one event. All displayed states accept finite prefixes; omitted transitions are absent. The initial states of the root, coordinator slot, and store slot are r_0r0, c_0c0, and b_0b0, respectively. For each ii, the root machine M_0M0 is

```
r0 --req(i)--> r1(i)
r1(i) --commit_ok(i)--> r2(i) --ack(i)--> rDone
r1(i) --commit_fail(i)--> r3(i) --fail(i)--> rDone
```

`req` is an input; the other root labels are outputs. Its language is

$$L_0 = \bigcup_i \mathrm{pref}\{\mathrm{req}(i)\,\mathrm{commit_ok}(i)\,\mathrm{ack}(i), \mathrm{req}(i)\,\mathrm{commit_fail}(i)\,\mathrm{fail}(i)\}.$$

L0 = i∪ pref{req(i) commit_ok(i) ack(i), req(i) commit_fail(i) fail(i)}.

Assume $\mathrm{Id} \neq \varnothing$Id ≠ ∅; equivalently add $\{\varepsilon\}${ε} explicitly when allowing an empty identifier type. The coordinator slot $B_{0,\,\mathrm{coord}}$B0,coord is

```
c0 --req(i)--> c1(i) --append(i)--> c2(i)
c2(i) --committed(i)--> c3(i) --ack(i)--> cDone
c2(i) --failed(i)--> c4(i) --fail(i)--> cDone
```

Its inputs are `req`, `committed`, and `failed`; its outputs are `append`, `ack`, and `fail`. Its language is the union of prefixes of the two displayed complete branches.

The store slot deliberately exposes its command protocol to the owning connector, which contains the substrate shell. This makes shell delivery part of the same guarded proof rather than an undeclared property of hidden commands. $B_{0,\,\mathrm{store}}$B0,store is

```
b0 --append(i)--> b1(i) --Cmd.write(i)--> b2(i)
b2(i) --write_ok(i)--> b3(i) --Cmd.fsync(i)--> b4(i)
b4(i) --fsync_ok(i)--> b5(i) --flush_ok(i)--> b6(i)
b6(i) --committed(i)--> bDone
b4(i) --fsync_fail(i)--> b7(i) --flush_fail(i)--> b8(i)
b8(i) --failed(i)--> bDone
```

Its inputs are `append`, `write_ok`, `fsync_ok`, and `fsync_fail`; its remaining events are outputs. Write $p_i = \mathrm{append}(i)\,\mathrm{Cmd.write}(i)\,\mathrm{write_ok}(i)\,\mathrm{Cmd.fsync}(i)$pi = append(i) Cmd.write(i) write_ok(i) Cmd.fsync(i). Then

$$L_{0,\mathrm{store}} = \bigcup_i \mathrm{pref}\{p_i\,\mathrm{fsync_ok}(i)\,\mathrm{flush_ok}(i)\,\mathrm{committed}(i),\ p_i\,\mathrm{fsync_fail}(i)\,\mathrm{flush_fail}(i)\,\mathrm{failed}(i)\}.$$

L0,store = i∪ pref{pi fsync_ok(i) flush_ok(i) committed(i), pi fsync_fail(i) flush_fail(i) failed(i)}.

Erasing command and response events yields the public protocol `append, flush_ok, committed` or `append, flush_fail, failed`, including every prefix. The local machines are $M_{\mathrm{coord}} = B_{0,\mathrm{coord}}$Mcoord = B0,coord and $M_{\mathrm{store}} = B_{0,\mathrm{store}}$Mstore = B0,store. Both edge maps $\lambda_{0,c}$λ0,c are identity maps on these declared slot alphabets; hiding occurs in the root connector observation, not in an inconsistent second edge map. Each graph's language is exactly its displayed prefix language, by enumeration of its initial path and two outgoing outcome branches. These are finite-control symbolic monitors, and are finite-state when Idld is finite.

8.3 Input scopes and concrete connector

Section titled "8.3 Input scopes and concrete connector"

The input-only scopes are

$$A_{0,\mathrm{coord}} = \{\varepsilon\} \cup \bigcup_i \{[\mathrm{req}(i)],\ [\mathrm{req}(i), \mathrm{committed}(i)],\ [\mathrm{req}(i), \mathrm{failed}(i)]\},$$

A0,coord = {ϵ} ∪ i∪{[req(i)], [req(i), committed(i)], [req(i), failed(i)]},

$$A_{0,\mathrm{store}} = \{\varepsilon\} \cup \bigcup_i \mathrm{pref}\{[\mathrm{append}(i), \mathrm{write_ok}(i), \mathrm{fsync_ok}(i)], [\mathrm{append}(i), \mathrm{write_ok}(i), \mathrm{fsync_fail}(i)]\}.$$

A0,store = {ϵ} ∪ i∪ pref{[append(i), write_ok(i), fsync_ok(i)], [append(i), write_ok(i), fsync_fail(i)]}.

Use these same scopes for the leaf realization obligations. They exclude wrong identifiers, duplicates, contradictory outcomes, and spontaneous response inputs. They do not express that a response follows a command output or that `committed` follows coordinator `append`; those causal facts require the connector and slot guarantees.

C_0C0 is a synchronous product of the two child witnesses and the shell of §8.5, with the following rules. A coordinator `append(i)` output synchronizes with the store's `append(i)` input. Store `committed(i)` and `failed(i)` outputs synchronize with the matching coordinator inputs. Each store command synchronizes with the shell's command input; each shell response synchronizes with the store's corresponding input. Unshared child events interleave. Child internal events interleave and are hidden. The environment supplies only the permitted root request. Shared events occur only when both participants have the transition; no event is fabricated or duplicated.

There is no arrival queue in this connector. Reception is checked at the paired states before each synchronous delivery. A deployment adding buffering must establish its distinct arrival/dispatch refinement (§4.4).

W_0W0 is the same finite-trajectory synchronization scheme over slot words and shell paths. Its root observation keeps coordinator `req`, `ack`, and `fail`, maps store `flush_ok` to `commit_ok` and `flush_fail` to `commit_fail`, and erases every other event. This is again a letter-or-erasure projection. Each root trajectory retains both component slices, including hidden commands, responses, and routed events, so it does not discard their causal correlation.

8.4 RLNF handler tables

Section titled "8.4 RLNF handler tables"

Both leaves start in `Idle` with an empty pending-output list. The coordinator's private states are `Idle`, `Await(i)`, and `Done`. Its enabled inputs and complete handler table are:

State	Enabled input	Next state	Output list
`Idle`	`req(i)`	`Await(i)`	`[append(i)]`
`Await(i)`	`committed(i)`	`Done`	`[ack(i)]`

State	Enabled input	Next state	Output list
`Await(i)`	`failed(i)`	`Done`	`[fail(i)]`
`Done`	none	—	—

The store's private states are `Idle`, `AwaitWrite(i)`, `AwaitFsync(i)`, and `Done`:

State	Enabled input	Next state	Output list
`Idle`	`append(i)`	`AwaitWrite(i)`	`[Cmd.write(i)]`
`AwaitWrite(i)`	`write_ok(i)`	`AwaitFsync(i)`	`[Cmd.fsync(i)]`
`AwaitFsync(i)`	`fsync_ok(i)`	`Done`	`[flush_ok(i), committed(i)]`
`AwaitFsync(i)`	`fsync_fail(i)`	`Done`	`[flush_fail(i), failed(i)]`
`Done`	none	—	—

The rows define enen exactly; no other state/input pair is enabled. Each row is a finite constructor computation and hence pure and terminating in the mathematical model. The elaboration, rather than an atomic multi-output transition, emits the output list one event at a time. In particular, after `flush_ok` but before `committed`, the pending list is `[committed(i)]` although the private state is already `Done`.

8.5 Substrate shell and pending-output simulations

Section titled "8.5 Substrate shell and pending-output simulations"

The shell $\mathrm{Sh}_{\mathrm{store}}$Shstore has initial state h_0h0 and transitions

```
h0 --Cmd.write(i)?--> hW(i) --write_ok(i)!--> hF(i)
hF(i) --Cmd.fsync(i)?--> hPending(i)
hPending(i) --fsync_ok(i)!--> hDone
hPending(i) --fsync_fail(i)!--> hDone
```

Question and exclamation marks indicate shell input and output roles, not extra labels. A response can occur only after its corresponding command, once, with the same identifier. A finite safety execution can stop at any state; stopping after command emission is a valid safety prefix, not evidence that every such execution is an admitted complete runtime service execution. The shell's physical interpretation is a hypothesis; H_{dur}Hdur applies to the successful flush transition after this write sequence. The safety argument does not assume that the shell eventually takes an enabled response. The operational progress argument uses H_{write}Hwrite to guarantee the matching write response, while leaving flush completion unconstrained.

The initialized coordinator simulation relates the following reachable elaboration configurations to monitor states:

Configuration	Monitor state
`(Idle, [])`	c_0c0
`(Await(i), [append(i)])`	$c_1(i)$c1(i)
`(Await(i), [])`	$c_2(i)$c2(i)
`(Done, [ack(i)])`	$c_3(i)$c3(i)
`(Done, [fail(i)])`	$c_4(i)$c4(i)
`(Done, [])`	c_{Done}cDone

For the store:

Configuration	Monitor state
`(Idle, [])`	b_0b0
`(AwaitWrite(i), [Cmd.write(i)])`	$b_1(i)$b1(i)
`(AwaitWrite(i), [])`	$b_2(i)$b2(i)
`(AwaitFsync(i), [Cmd.fsync(i)])`	$b_3(i)$b3(i)
`(AwaitFsync(i), [])`	$b_4(i)$b4(i)
`(Done, [flush_ok(i), committed(i)])`	$b_5(i)$b5(i)
`(Done, [committed(i)])`	$b_6(i)$b6(i)
`(Done, [flush_fail(i), failed(i)])`	$b_7(i)$b7(i)
`(Done, [failed(i)])`	$b_8(i)$b8(i)
`(Done, [])`	b_{Done}bDone

Each input row matches its monitor input transition; each output–drain rule matches exactly one output arrow. The tables enumerate all reachable configurations. Hence they prove both leaf realization inclusions, including every intermediate prefix and both outcomes. The reverse graph–to–table step correspondence also gives equality of these mathematical leaf languages. Relative reception holds at c_0, c_2, b_0, b_2, b_4c0, c2, b0, b2, b4 for exactly their permitted inputs. Pending–output states correspond to output–only monitor states, so no permitted input is waived. Initial states are nonempty, and the two leaves have disjoint private state owned by root $0$0.

8.6 Causal delivery, wiring, decomposition, and root soundness

Section titled “8.6 Causal delivery, wiring, decomposition, and root soundness”

We prove guarded $\mathrm{Wire}(0)$Wire(0) for arbitrary admissible witnesses refining the two fixed slots, not just the tables. The shell and synchronization rules are fixed operands. Consider a supposed first delivery outside a declared input scope. Before it, every child input history is in scope, so its executed prefix and any output extending that prefix are governed by its slot inclusion: outputs cannot be hidden by the input restriction.

The external request is single–use and matching by the root environment premise. A coordinator `append(i)` can occur only after that `req(i)`, once. The store can issue `Cmd.write(i)` only after receiving `append(i)`; the shell can return `write_ok(i)` only after receiving that command. A store `Cmd.fsync(i)` can occur only after the matching write response, and the shell can supply at most one matching flush outcome. Finally, a store `committed(i)` or `failed(i)` can occur only after its matching flush event, itself after `append(i)`. Thus coordinator response delivery is after its append output has drained and is single–use, matching, and non–spontaneous.

To make the first–invalid–input argument fully explicit, suppose for contradiction that some composite step delivers a first input prefix violating either $A_{0,\,\mathrm{coord}}$A0,coord or $A_{0,\,\mathrm{store}}$A0,store:

1. *First invalid input delivered to the coordinator:* The only permitted inputs in $A_{0,\,\mathrm{coord}}$A0,coord are an initial single $\mathrm{req}(i)$req(i), followed by at most one matching $\mathrm{committed}(i)$committed(i) or $\mathrm{failed}(i)$failed(i). A violation would require: (a) a spurious or duplicate $\mathrm{req}(j)$ req(j), which is barred by the root environment premise; (b) a spontaneous $\mathrm{committed}(i)$committed(i) or $\mathrm{failed}(i)$failed(i) prior to emitting $\mathrm{append}(i)$append(i), which contradicts the store slot $B_{0,\,\mathrm{store}}$B0,store (which requires $\mathrm{append}(i)$append(i) before any response); or (c) an outcome with mismatched identifier $j \neq i$j = i or a contradictory second outcome, both barred by the single–identifier execution of $B_{0,\,\mathrm{store}}$B0,store and shell synchronization.
2. *First invalid input delivered to the store:* Under the fixed shell $\mathrm{Sh}_{\mathrm{store}}$Shstore and command synchronization, admissible delivered input histories to the store consist of an initial $\mathrm{append}(i)$append(i), followed by $\mathrm{write_ok}(i)$write_ok(i) strictly after $\mathrm{Cmd.write}(i)$ Cmd.write(i), and finally $\mathrm{fsync_ok}(i)$fsync_ok(i) or $\mathrm{fsync_fail}(i)$fsync_fail(i) strictly after $\mathrm{Cmd.fsync}(i)$Cmd.fsync(i). A violation would require: (a) a spurious or duplicate $\mathrm{append}(j)$append(j), which contradicts coordinator slot conformance (the coordinator emits $\mathrm{append}(i)$append(i) at most once, strictly in response to $\mathrm{req}(i)$req(i)); or (b) an unsolicited, out–of–order, or duplicate shell

response, which contradicts the transition structure of $\mathrm{Sh}_{\mathrm{store}}$Shstore (which transitions to response states only upon receiving the corresponding command from the store).

Every branch of this finite case split contradicts the causal prefixes of the component slots and shell. All delivered histories therefore lie in the stated AA sets, including the intrinsic leaf scopes.

This argument also checks readiness: the routed `append` reaches the store's initial input state; each shell response reaches the corresponding store input state; and the store result reaches coordinator $c_2(i)$c2(i), not its pending–append state. Relative reception of the admitted witnesses supplies the matching input transition. Hidden internal source steps cannot change the deterministic monitor state. Thus the proof does not rely on suppressing an invalid delivery by composing already restricted machines. If a raw witness had a first bad delivery, the same prefix argument would exclude it.

For routing adequacy, induct on concrete composite steps. An unshared transition extends one component slice; a synchronization extends both participating slices with the shared event; a shell step also extends the shell path. The retained joint record is legal for W_0 W0 and has exactly the root observation defined in §8.3. Combining this slicing invariant with the proved input scopes gives $\mathrm{Delivery}_0$ Delivery0 and the restricted inclusion required by $\mathrm{Wire}(0)$Wire(0).

For $\mathrm{Dec}(0)$Dec(0), take any legal trajectory whose slices belong to the slot languages. If no root request has occurred, its root observation is empty. After `req(i)`, the coordinator emits `append(i)` at most once. The store's selected branch can contribute either `commit_ok(i)` followed eventually, if the trajectory continues, by coordinator `ack(i)`, or `commit_fail(i)` followed by `fail(i)`. The coordinator cannot acknowledge before receiving the store result, which follows the matching flush event. No other root events are possible. Hence every root observation is in L_0L0, including truncations at all hidden or visible intermediate events.

The two edge refinements are immediate because restriction selects a sublanguage $(\mathrm{tr}(M_c)\restriction_{A_{0,c}} \subseteq \mathrm{tr}(M_c)$tr(Mc)↾A0,c⊆ tr(Mc)), the edge maps $\lambda_{0,c}$λ0,c are identities, and the local component machines are defined by $M_c = B_{0,c}$Mc = B0,c. The simulations establish realization; the delivery proof justifies the shared input scopes; the product has a nonempty initial state and accepts the permitted initial request. Root satisfaction is $\mathrm{tr}(M_0) = L_0 \subseteq \Phi_0$tr(M0) = L0 ⊆ Φ0, where Φ_0Φ0 is a necessary safety predicate requiring that every `ack(i)` be preceded by `commit_ok(i)` and every `fail(i)` by `commit_fail(i)` (with request precedence and branch mutual exclusivity ensured by L_0L0). Theorem 1 therefore yields the complete root trace inclusion. Under hypothesis H_{dur}Hdur, emitting `ack(i)` establishes that the successful write has been durably flushed to persistent storage, with the overall root guarantee formally supported by the combined environment, shell, and runtime premises in H_0H0.

A spontaneous–output store serves as an informative negative witness: even if receptive and properly initialized, emitting an initial `committed(i)` violates its slot contract and is decisively rejected by the universal wiring guard.

8.6.1 Readiness, non–silent progress, and external wait

Section titled “8.6.1 Readiness, non–silent progress, and external wait”

We evaluate operational readiness and non–silent response on the synchronized composite execution witness $N_0 = C_0[N_{\mathrm{coord}}, N_{\mathrm{store}}, \mathrm{Sh}_{\mathrm{store}}]$ N0 = C0[Ncoord, Nstore, Shstore].

- **Pending duty (P_dPd):** Duty dd is pending in all composite states following external input `req(i)` prior to the emission of root `ack(i)` or `fail(i)`.
- **Transient write wait ($W_{d,\mathrm{write}(i)}$Wd,write(i)):** A genuine write wait is the composite configuration with coordinator `(Await(i), [])`, store `(AwaitWrite(i), [])`, and shell $hW(i)$hW(i) following the synchronized `Cmd.write(i)`. Here $W_{d,\mathrm{write}(i)}$Wd,write(i) holds because the system awaits the external write response. Under premise H_{write}Hwrite, this wait is transient; it is excluded from persistent waiting (E_d^{∞}Ed∞).
- **Genuine persistent flush wait ($W_{d,\mathrm{flush}(i)}$Wd,flush(i)):** A genuine flush wait is the composite configuration with coordinator `(Await(i), [])`, store `(AwaitFsync(i), [])`, and shell $hPending(i)$hPending(i) following `Cmd.fsync(i)`. The root contract explicitly permits this wait to persist indefinitely:

$$E_d^{\infty} = \{\text{the matching flush prerequisite flush}(i)\}.$$
Ed∞ = {the matching flush prerequisite flush(i)}.

- **Owned continuations:** Configurations with pending outputs—such as store `(Done, [flush_ok(i), committed(i)])`, store `(Done, [committed(i)])`, or coordinator `(Done, [ack(i)])`—are **not** external waits. They represent owned internal work that must be drained.
- **Inspection exposure:** Correlated composite wait states are exposed through the inspection projection, identifying the outstanding duty and prerequisite. They indicate pending status, not durable completion or failure.

The composite transition system over the single request is acyclic: every owned handler execution and output drain advances along a finite directed acyclic graph. At every pending configuration where neither $W_{d,\text{write}(i)}$Wd,write(i) nor $W_{d,\text{flush}(i)}$Wd,flush(i) holds, an owned continuation transition is enabled: coordinator pending outputs (`append(i)`, `ack(i)`, `fail(i)`) synchronize with the receptive initial store or emit externally; store pending outputs (`Cmd.write(i)`, `Cmd.fsync(i)`, `flush_ok(i)`, `committed(i)`, etc.) synchronize with the receptive shell or coordinator; and once outputs drain after a shell response (`write_ok(i)`, `fsync_ok(i)`, or `fsync_fail(i)`), the corresponding store handler is enabled by proved relative reception. Thus $P_d(s) \wedge \neg\exists e\, W_{d,e}(s) \Longrightarrow \exists s'.\ s \xrightarrow{\text{owned}(d)} s'$Pd(s) ∧ ¬∃e Wd,e(s) ⟹ ∃s′. s owned(d)

---

s′, establishing condition (Ready)(Ready).

We evaluate complete maximal executions in $\text{CExec}_0(N_0, \omega)$CExec0(N0, ω) under the following explicit scheduling premise:

> **Execution Premise H^0_{exec}Hexec0:** For every boundary–owned transition on the example's finite per–request continuation, if that transition remains continuously enabled while duty dd is outstanding, it is not postponed forever.

Premise H^0_{exec}Hexec0 is a per–transition weak–fairness condition over the finite continuation of this specific example. It is not a general theorem about host schedulers, nor does it assert universal progress for cyclic systems.

Proof of NSNS for the worked example:

1. **Finite owned paths:** Because the owned control graph is finite and acyclic, the system cannot execute infinitely many boundary–owned steps for a single request.
2. **Exclusion of internal hangs:** Under H^0_{exec}Hexec0, execution cannot remain permanently in a pending non–wait configuration, nor can it abandon an undrained pending output buffer.
3. **Write resolution:** By premise H_{write}Hwrite, the substrate shell eventually emits `write_ok(i)`, ending the transient write wait. The subsequent finite owned continuation issues `Cmd.fsync(i)`.
4. **Outcome or inspectable flush wait:** If the substrate shell resolves the flush with `fsync_ok(i)` or `fsync_fail(i)`, the remaining finite owned suffix executes under H^0_{exec}Hexec0, emitting root `ack(i)` or root `fail(i)` (FD_dFDd). Otherwise, when the substrate emits no flush response, no boundary–owned transition in Owned_0Owned0 is enabled (even though the static shell graph has outgoing response edges from $hPending(i)$hPending(i)), so the complete execution stutters indefinitely in configuration $hPending(i)$ hPending(i), satisfying $W_{d,\text{flush}(i)}$Wd,flush(i). Because $\text{flush}(i) \in E_d^{\infty}$flush(i) ∈ Ed∞, this execution satisfies $\exists e \in E_d^{\infty}.\ FGW_{d,e}$ ∃e ∈ Ed∞. FGWd,e. Persistent write waiting is excluded by H_{write}Hwrite.

Therefore, every complete execution satisfies $FD_d \vee \exists e \in E_d^{\infty}.\ FGW_{d,e}$FDd ∨ ∃e ∈ Ed∞. FGWd,e, discharging condition (NS)(NS).

8.7 Premature–commit regression and non–progress regressions

Section titled "8.7 Premature–commit regression and non–progress regressions"

To demonstrate the discriminating power of the calculus, we evaluate candidate mutant implementations against the established safety and operational obligations.

8.7.1 Premature–acknowledgement safety regression

Section titled "8.7.1 Premature–acknowledgement safety regression"

Consider a faulty store handler `Idle + append(i) -> Done, [committed(i)]`, with the other initial inputs unchanged. Its trace `append(i), committed(i)` is rejected by the fixed store graph: after `append(i)` the next required visible output is `Cmd.write(i)`, and the projected public protocol also requires `flush_ok(i)` before `committed(i)`. The input history `[append(i)]` is admissible, so input restriction does not remove the counterexample.

Composed with the coordinator, this witness produces the root trace `req(i), ack(i)`. It is not in L_0L0, because `ack(i)` lacks its preceding `commit_ok(i)`. The expected rejection is stated by the unchanged graphs, not copied from the faulty implementation. This run does not contradict Theorem 1: the faulty store fails realization/refinement, so the universal wiring guard cannot be instantiated for it.

8.7.2 Operational non-progress regressions

Section titled “8.7.2 Operational non-progress regressions”

The following structural scenarios illustrate how operational obligations (Ready)(Ready) and (NS)(NS) reject invalid behaviors that satisfy finite-safety trace containment:

1. **Silent terminal loss:** Replace the coordinator request handler by `Idle + req(i) -> Done, []`. The finite trace language $\{\epsilon, \mathrm{req}(i)\}$ {ϵ,req(i)} satisfies safety inclusion, but the response duty is abandoned. The completed configuration has P_dPd, no D_dDd, no WW, and no enabled owned transition. It is decisively rejected by ReadyReady and NSNS.
2. **Enabled-but-untaken outcome loop:** Consider an operational witness whose runtime admits an internal self-loop while a terminal outcome transition remains continuously enabled. This execution satisfies finite trace inclusion and condition (Ready)(Ready), but fails (NS)(NS) because the duty is never discharged. Premise H^0_{exec}Hexec0 excludes this behavior for the acyclic example, but a general runtime lacking fairness must treat this regression as a real failure mode.
3. **Resolved-but-undrained output:** Following `fsync_ok(i)`, the store transitions to `Done` with pending list `[flush_ok(i), committed(i)]`. If the runtime halts output draining, the prerequisite has resolved, so $W_{d,\,\mathrm{flush}(i)}$Wd,flush(i) ceases to hold. The execution stalls with an outstanding duty and no wait explanation, violating NSNS.
4. **Persistent flush wait accepted:** A complete execution that issues `Cmd.fsync(i)` and remains permanently in $hPending(i)$hPending(i) satisfies NSNS via its second disjunct, because $\mathrm{flush}(i) \in E_d^\infty$flush(i) ∈ Ed∞. The system correctly reports the duty as pending without manufacturing an artificial timeout or error.
5. **Parent-owned mutual waiting:** Suppose two internal child components deadlock waiting for each other. Because both components lie within the parent boundary’s realization, their mutual dependency is internal work. The parent cannot classify this internal blockage as an external wait ($W_{d,\,e}$Wd,e). The complete stalled execution fails ReadyReady and NSNS.
6. **Strict contract with persistent wait:** Consider a strict terminal-response contract with $E_d^\infty = \varnothing$Ed∞ = ∅. A complete execution that stalls permanently in $hPending(i)$hPending(i) fails NSNS, whereas an execution with a transient wait followed by terminal acknowledgement passes. This confirms that the wait policy E_d^∞Ed∞ prevents indefinite stalling from discharging a strict contract, while still permitting legitimate transient waits.

Valid input-only operations, ordinary finite prefixes, and terminating computations with finite delay remain positive non-regressions.

8.8 Inspection and proof reuse

Section titled “8.8 Inspection and proof reuse”

The frontier $\{\mathrm{coord}, \mathrm{store}\}$coord,store is root-covering and displays both slot protocols. Realization and admission evidence at this frontier transfers finite-safety assurance upward to the root via Corollary 1. Inspecting only the coordinator omits the store path and cannot justify root safety.

This case study illustrates the two human governance tasks formulated in §3.9:

- **Upstream authoring task:** A system architect evaluating whether `ack(i)` requires stronger durability guarantees examines the root intent, the store slot contract, and hypothesis H_{dur}Hdur at the storage substrate boundary. The decision does not require reading or modifying the coordinator’s internal handler definitions.
- **Downstream assessment task:** When inspecting an unresolved request, an operator examines the exposed state to distinguish a genuine external wait at the storage flush from an undrained internal output queue or an internal component stall, identifying

precisely which environmental assumption or implementation step governs progress.

These scenarios represent concrete governance tasks; they are evaluation designs, not completed empirical human–subject trials.

Suppose the store's internal representation is modified while its component model, shell protocol, input scope, and slot contract remain unchanged. Once the new handler realization and admission obligations are verified, Corollary 2 allows immediate reuse of existing proofs for $\mathrm{Wire}(0)$Wire(0), $\mathrm{Dec}(0)$Dec(0), coordinator compliance, and root satisfaction. Reusing operational progress proofs additionally requires verifying that H_{write}Hwrite and runtime scheduling premises remain intact. If an update instead alters the substrate crash model, command protocol, or delivery buffering, the affected operands are updated and re–verified accordingly.

9. Related Work

Section titled "9. Related Work"

Transition–system refinement and interfaces. Lynch and Tuttle (1989) separate input, output, and internal actions and make execution composition explicit. De Alfaro and Henzinger (2001) distinguish environmental inputs from component outputs in interface compatibility. Our framework adopts these distinctions within a finite–trace safety setting, pairing trace containment with explicit relative reception and causal delivery obligations to ensure receptive composition. Abadi and Lamport (1991, 1995) study refinement mappings and composition of specifications; Lamport (1994) makes stuttering and temporal interpretation explicit. Here the observation map is restricted enough to prove the needed finite–prefix law directly, without importing a liveness theorem.

Refinement calculi and reactive modeling. UTP designs distinguish feasibility and termination–sensitive meanings from arbitrary relations (Woodcock and Cavalcanti, 2004). Circus separates data and process refinement (Cavalcanti, Sampaio, and Woodcock, 2003), while Event–B organizes initialization, invariants, and refinement obligations (Abrial, 2010). RLNF identifies a structured, pure, terminating handler fragment suitable for automated synthesis and verification. The extensive systems verified by IronFleet (Hawblitzel et al., 2015) highlight the importance of connecting high–level state–machine models to executable distributed code; our calculus formalizes this connection via explicit translation premises and typed runtime obligations.

Shared services. Linearizability provides concurrent operations with a legal sequential interpretation respecting real–time order (Herlihy and Wing, 1990; Filipovic´et al., 2010). The core calculus captures shared services through single–owner state and explicit parent–mediated serialization, establishing aggregate correctness across concurrent clients. The serialization observed in the shared–counter example is an architectural property of the chosen connector skeleton, rather than an inescapable law of shared mutable state.

Evidence and agent–oriented assurance. Rushby's Evidential Tool Bus (2005) and the implementations and analyses of Cruanes et al. (2013, 2014) address heterogeneous verification evidence and its logical dependencies. Assurance 2.0's treatment of defeaters distinguishes challenges to claims, arguments, and evidence (Bloomfield, Netkachova, and Rushby, 2024). CLARISSA develops foundations and automation for assurance cases (Varadarajan et al., 2023); PLACIDUS formalizes assurance–case development for software product lines (Murphy et al., iFM 2024); AssureCode connects developer intent, specifications, and analysis evidence in agentic coding (Murphy et al., 2026). Our contribution integrates these assurance principles with an operational refinement calculus: while structured assurance frameworks organize arguments and defeaters (Bloomfield, Netkachova, and Rushby, 2024; Murphy et al., 2024, 2026), our development DAG provides the formal trace semantics, obligation taxonomy, and typed gates required to verify agent–authored systems.

Provenance, incrementality, and counterexamples. Our provenance and cutoff mechanisms draw upon foundational work in database provenance (Green et al., 2007) and modern build systems (Mokhov et al., 2018). Positive Boolean provenance provides an explanatory model for intent activation paths, rather than requiring an expensive runtime enumeration of all minimal support powersets. Our diagnostic taxonomy builds upon counterexample–guided abstraction refinement (Clarke et al., 2003) to clearly separate abstract model defects from concrete runtime violations.

10. Limitations and Open Problems

Section titled "10. Limitations and Open Problems"

The core calculus proves finite visible–trace safety for its admitted operational fragment. Finite trace inclusion alone permits silence and internal divergence; the operational obligations ReadyReady and NSNS reject these covered failure modes separately. Genuine, indefinitely unresolved external prerequisites are admitted only where the accepted contract's wait policy explicitly allows them ($e \in E_d^\infty$ e ∈ Ed∞), and such states remain visible as pending. This operational condition does not guarantee eventual success, a terminal

response under arbitrary environment failure, or hard real-time deadlines. General progress-preserving abstraction and component substitution require dedicated temporal refinement theorems; the finite trace safety theorems do not supply them.

The example's scheduling premise H^0_{exec}Hexec0 is an explicit per-use weak-fairness assumption on a finite acyclic continuation, not a proved property of a host operating system scheduler. Weak fairness concerns eventual continuous enabling, whereas strong fairness concerns repeated enabling; neither follows from finite trace containment or terminating leaf handlers. No general fairness theorem is proved.

RLNF excludes hidden effects and unproved within-handler divergence, but does not prevent connector-level communication deadlocks if slot contracts permit circular waits. State-indexed reception restricts deployment inputs; it is not global input-enabledness. Sharing restrictions exclude unmediated mutable aliases and do not provide a general concurrent-history substitution theorem. Adding those behaviors requires new boundary semantics and proofs rather than a broader interpretation of an existing inclusion.

Symbolic state and data domains may be infinite. Deterministic-monitor simulation completeness is an existence theorem, not an algorithm for automatically synthesizing invariants or ranking functions. Context consistency, physical substrate fidelity, source-to-elaboration translation, and publication correctness can remain trusted premises. Contract precision limits what a parent can establish: a Top slot cannot carry a nontrivial durability promise by itself.

The worked example is a single-request model, not a production storage implementation or crash-recovery protocol. Furthermore, runtime scheduling correspondence and interactive visual inspection interfaces are not mechanized in this paper. The paper makes no empirical claim that the workflow reduces agent error rates, cognitive load, review cost, or proof effort. Demonstrating those benefits requires implementations, representative tasks, and measured comparisons with explicitly scoped acceptance criteria.

11. Conclusion

Section titled "11. Conclusion"

A refinement DAG separates what a boundary promises, how its children establish that promise, and why the available evidence applies. In service of five root purposes—realizing committed behavior, preserving authority over governing meaning, keeping justified change local, reducing human governance context, and keeping assurance accountable to evidence—the calculus establishes a rigorous boundary for agent-authored systems.

RLNF supplies executable leaf semantics, homomorphic observations connect traces across interfaces, deterministic monitors support complete simulation reasoning, and single-owner state locates interaction obligations. Guarded wiring closes the composition seam, while protected contracts and the incremental frame theorem enable exact-operand evidence reuse across versioned evolution without silently changing accepted meaning. Separately verified operational obligations govern input reception, operational readiness, and contract-relative non-silent response, distinguishing permitted terminal outcomes from genuine, inspectable external waits. The resulting guarantee is conditional, compositional, and explicit about its operational, environmental, and trusted boundaries.

Appendix A. Proof Details

Section titled "Appendix A. Proof Details"

A.1 Theorem 1: full conditional DAG induction

Section titled "A.1 Theorem 1: full conditional DAG induction"

Fix a root rr and a world ωω with $H_r(\omega)$Hr(ω). By definition of transitive dependency closure and the declared scope-entailment premises, every local obligation and admission judgment used below holds at this same ωω. Let $d(v)$d(v) be the maximum length of a path from vv to a leaf; finiteness and acyclicity give $d(c) < d(v)$d(c) < d(v) on every child edge. Abbreviate $Q_v = \mathrm{tr}(N_v(\omega))$Qv = tr(Nv(ω)) and $L_v = \mathrm{tr}(M_v(\omega))$Lv = tr(Mv(ω)).

First, $Q \subseteq Q'$Q ⊆ Q′ implies $Q \restriction_A \subseteq Q' \restriction_A$Q↾A⊆Q′↾A, since membership retains the same input predicate. Direct image under λ^*λ∗ preserves inclusion, giving $T_{v,c}(Q) \subseteq T_{v,c}(Q')$Tv,c(Q) ⊆ Tv,c(Q′). This uses neither operational composition of sets nor output filtering.

Proceed by induction on $d(v)$d(v). If $d(v) = 0$d(v) = 0, the scoped executable witness is the leaf runtime interpretation. Trans_ℓTransℓ, when needed, gives its inclusion in $\mathrm{tr}(\mathrm{elab}(D_\ell)) \restriction_{A_\ell}$ tr(elab(Dℓ)) ↾Aℓ, and leaf realization gives inclusion in L_ℓLℓ. This is $\mathrm{Impl}(\ell)$Impl(ℓ). Initial states and reception are supplied by admission, not inferred from inclusion.

For an internal vv, assume $Q_c \subseteq L_c$Qc ⊆ Lc for every child. Then for each edge,

$T_{v,c}(Q_c) \subseteq T_{v,c}(L_c) \subseteq L_{v,c}$.Tv,c(Qc) ⊆ Tv,c(Lc) ⊆ Lv,c.

The actual N_cNc is admitted by Condition 6, so the tuple of these actual witnesses satisfies the entire guard of Condition 5. Its conclusion gives both delivery and

$Q_v \subseteq K_v((T_{v,c}(Q_c))_c)$.Qv ⊆ Kv((Tv,c(Qc))c).

Lemma 1 and the previous bounds yield

$$Q_v \subseteq K_v((T_{v,c}(Q_c))_c) \subseteq K_v((T_{v,c}(L_c))_c) \subseteq K_v((L_{v,c})_c) \subseteq L_v,$$
$$\mathrm{Qv} \subseteq \mathrm{Kv}((\mathrm{Tv,c}(\mathrm{Qc}))\mathrm{c}) \subseteq \mathrm{Kv}((\mathrm{Tv,c}(\mathrm{Lc}))\mathrm{c}) \subseteq \mathrm{Kv}((\mathrm{Lv,c})\mathrm{c}) \subseteq \mathrm{Lv},$$

where the last inclusion is decomposition. Condition 3 then gives $L_v \subseteq [\![j]\!]$Lv ⊆ [[j]] for every $j \in A(v)$j ∈ A(v), so $Q_v \subseteq \Phi(v)$Qv ⊆ Φ(v). When $A(v)$A(v) is empty, $\Phi(v)$Φ(v) defaults to the entire typed trace universe. In particular, $Q_r \subseteq \Phi(r)$Qr ⊆ Φ(r). Since ωω was arbitrary under H_rHr, the theorem holds universally across all models of the root context. Single-owner encapsulation and typed runtime interpretations ensure that the execution witnesses correspond directly to the declared physical system instances. □□

A.2 Corollary 1: full frontier induction

Section titled “A.2 Corollary 1: full frontier induction”

Let FF be a root-covering antichain. Every root has a descendant in FF. For $v \in \mathrm{Anc}(F) \setminus F$v ∈ Anc(F) ∖ F and child cc, choose a root-to-vv path and extend the $v \to c$v → c path to a leaf. No node of FF can be strictly above vv on this path, because vv has a descendant in FF and that would violate the antichain property. Coverage therefore supplies a cut node at or below cc. Thus $c \in \mathrm{Anc}(F)$c ∈ Anc(F), and the recursive definition of U_FUF is closed on the upper region.

Fix a world satisfying the full contexts of frontier realization and all upper obligations. Induct in increasing child rank, stopping at FF. At $f \in F$f ∈ F, $Q_f \subseteq U_F(f) = L_f$Qf ⊆ UF(f) = Lf is the frontier premise. At an upper internal node, assume $Q_c \subseteq U_F(c) \subseteq L_c$Qc ⊆ UF(c) ⊆ Lc for every child. The right half, monotonicity, edge refinement, and decomposition give

$$U_F(v) = K_v((T_{v,c}(U_F(c)))_c) \subseteq K_v((T_{v,c}(L_c))_c) \subseteq K_v((L_{v,c})_c) \subseteq L_v.$$
$$\mathrm{UF(v)} = \mathrm{Kv}((\mathrm{Tv,c}(\mathrm{UF(c)}))\mathrm{c}) \subseteq \mathrm{Kv}((\mathrm{Tv,c}(\mathrm{Lc}))\mathrm{c}) \subseteq \mathrm{Kv}((\mathrm{Lv,c})\mathrm{c}) \subseteq \mathrm{Lv}.$$

The same bounds imply the slot guards for the actual witnesses. Their separately assumed admission permits wiring instantiation, yielding

$$Q_v \subseteq K_v((T_{v,c}(Q_c))_c) \subseteq K_v((T_{v,c}(U_F(c)))_c) = U_F(v).$$
$$\mathrm{Qv} \subseteq \mathrm{Kv}((\mathrm{Tv,c}(\mathrm{Qc}))\mathrm{c}) \subseteq \mathrm{Kv}((\mathrm{Tv,c}(\mathrm{UF(c)}))\mathrm{c}) = \mathrm{UF(v)}.$$

Root satisfaction supplies the final inclusion in $\Phi(r)$Φ(r). The induction accommodates arbitrary cut configurations and shared state-free components by following well-founded partial ordering over the DAG, with actual-witness admission independently verified at the frontier boundary. □□

A.3 Packet Decomposition

Section titled “A.3 Packet Decomposition”

Consider a finite path from $(s_0, [])$(s0,[]) in $\mathrm{elab}(D)$elab(D). Every input step starts with an empty pending list. It selects exactly one enabled handler result (s', w)(s′,w), after which only the successive heads of ww can be emitted until the list empties. Induction on the number of input steps partitions the path into complete packets and, possibly, one last input with an initial segment of its output word. Because that remaining word is finite, drain it without any further input. The resulting trace is a member of $\mathrm{Packets}(D)$Packets(D) extending the original trace. A path with no input is the zero-step path and has trace ϵϵ. Thus the left side is contained in the right.

Conversely, take a packet word from an enabled finite handler chain. Starting at $(s_0, [])$(s0,[]), apply each chain input rule and then drain its listed outputs. The handler's next state is exactly the next chain state, so this constructs an execution of the whole word. For any prefix of the word, stop this execution after the corresponding event. Empty outputs require no drain transitions; the empty prefix requires no transitions. Thus every prefix is an elaborated trace. □□

The lemma accounts for intermediate prefixes occurring between successive output emissions, precisely reflecting microstep evaluation while preserving the invariant that input dispatch occurs only when pending output buffers are exhausted.

A.4 Prefix Commutation

Section titled "A.4 Prefix Commutation"

For $u \preceq t$u ⪯ t, choose zz with $t = uz$t = uz. Homomorphic extension gives $\lambda^*(t) = \lambda^*(u)\lambda^*(z)$λ∗(t) = λ∗(u)λ∗(z), so $\lambda^*(u) \preceq \lambda^*(t)$ λ∗(u) ⪯ λ∗(t). This proves $\lambda^*(\mathrm{pref} L) \subseteq \mathrm{pref}(\lambda^* L)$λ∗(pref L) ⊆ pref(λ∗L).

For the converse take $p \preceq \lambda^*(t)$p ⪯ λ∗(t) with $t = a_1 \cdots a_n \in L$t = a1 ⋯ an ∈ L. If $p = \epsilon$p = ϵ, use the source prefix ϵϵ. Otherwise let $k = |p|$k = |p|. Each source letter contributes zero or one target letter. Scanning $a_1, \ldots, a_n$a1, … , an, the count of contributed letters begins at zero, ends at least at kk, and increases by at most one. Hence some first index jj reaches exactly kk. The image of $a_1 \cdots a_j$a1 ⋯ aj is the first kk letters of $\lambda^*(t)\lambda^*(t)$, namely pp. That source prefix belongs to $\mathrm{pref} L$pref L. If LL is empty, neither side has a witness. This proves equality in every case. A multi-letter image can jump over kk, which is exactly the obstruction in $a \mapsto xy$a ↦ xy. □□

A.5 Deterministic Contract Simulation Equivalence

Section titled "A.5 Deterministic Contract Simulation Equivalence"

Let BB have one initial state q_0q0, no hidden transitions, partial deterministic ββ, and every reachable state accepting. For a word ww admitted by BB, let $\hat{\beta}(q_0, w)$β

(q0,w) denote its unique reached state. Extend λλ by erasing source internal labels.

Suppose initialized simulation RR exists. For a finite source execution, start with its initial state related to q_0q0. Induct over its transitions. An erased action leaves the target state unchanged; a visible image follows the required target transition. At each step the new states are related. Thus the target admits precisely the concatenation of non-erased images, which is λ^*λ∗ of the source visible trace. All finite source traces are included.

Conversely assume inclusion. Define $(s, q) \in R$(s,q) ∈ R iff there exists a finite source execution ρρ ending at ss such that

$q = \hat{\beta}(q_0, \lambda^*(\mathrm{vis}(\rho)))$.q = β

(q0, λ∗(vis(ρ))).

Inclusion makes this value defined for every witnessing execution. Every source initial state has the zero-step execution, so initialization holds. Let $(s, q) \in R$(s,q) ∈ R be witnessed by ρρ, and let $s \xrightarrow{a} s'$s a

---

s′. The extended execution ρaρa is a source execution. If aa is internal or $\lambda(a) = \epsilon$λ(a) = ϵ, its mapped trace is unchanged, giving $(s', q) \in R$(s′,q) ∈ R. Otherwise write $b = \lambda(a)$b = λ(a). Inclusion admits $\lambda^*(\mathrm{vis}(\rho))b\lambda^*(\mathrm{vis}(\rho))$b. Determinism and the unique initial state

mean its prefix reaches exactly qq, so $\beta(q, b)$β(q, b) is defined, say q'q′, and the extended execution witnesses $(s', q') \in R$(s′, q′) ∈ R. These are all simulation clauses.

The construction allows multiple target states to relate to a source state through history-dependent executions. For scoped input languages, the equivalence applies directly to the operational machine $\mathrm{Restrict}_A(N)$RestrictA(N) via restriction equality. □□

A.6 Finite-control progress proof for the durable acknowledgement example

Section titled “A.6 Finite-control progress proof for the durable acknowledgement example”

We formalize the four-step progress proof for the composite execution witness N_0N0 under scheduling premise H^0_{exec}Hexec0 and write premise H_{write}Hwrite.

Let $\pi \in \mathrm{CExec}_0(N_0, \omega)$π ∈ CExec0(N0, ω) be a maximal execution, and let request `req(i)` occur at step k_0k0, activating duty dd. The composite reachable state space is finite and acyclic. Every owned transition is labeled by an action in Owned_0Owned0.

1. **Absence of infinite owned executions:** Since the reachable control graph contains no directed cycles of owned transitions, ππ cannot contain infinitely many consecutive owned transitions.
2. **Readiness on pending states:** By the configuration case split in §8.6.1, every reachable pending configuration ss where neither $W_{d,\,\mathrm{write}(i)}$Wd,write(i) nor $W_{d,\,\mathrm{flush}(i)}$Wd,flush(i) holds enables an owned continuation transition $s \xrightarrow{\mathrm{owned}(d)} s'$s owned(d) s′.
3. **Write phase:** If ππ reaches write wait $W_{d,\,\mathrm{write}(i)}$Wd,write(i) (shell state $hW(i)$hW(i)), premise H_{write}Hwrite ensures that the matching response `write_ok(i)` is emitted after finite delay. Under H^0_{exec}Hexec0, the store cannot indefinitely postpone dispatching this enabled response.
4. **Flush phase and disjunctive closure:** Following write completion, the acyclic continuation enables `Cmd.fsync(i)`. Under H^0_{exec}Hexec0, this command is synchronized, reaching flush wait configuration $hPending(i)$hPending(i) where $W_{d,\,\mathrm{flush}(i)}$Wd,flush(i) holds.
   - Case 4a: If the substrate shell emits `fsync_ok(i)` or `fsync_fail(i)`, the enabled store handler transitions to state `Done` with pending outputs. Under H^0_{exec}Hexec0, each enabled drain transition is executed in finite steps, reaching root emission of `ack(i)` or `fail(i)`. This witnesses terminal outcome D_dDd, satisfying the first disjunct FD_dFDd.
   - Case 4b: If the substrate shell emits no response, no owned transition is enabled. The execution stutters indefinitely in $hPending(i)$hPending(i), where $W_{d,\,\mathrm{flush}(i)}$Wd,flush(i) holds globally thereafter. Because $\mathrm{flush}(i) \in E^{\infty}_d$flush(i) ∈ Ed∞, this witnesses the second disjunct $\exists e \in E^{\infty}_d.\, FGW_{d,e}$∃e ∈ Ed∞. FGWd,e.

Therefore, $\pi \vDash G(P_d \Rightarrow [FD_d \vee \exists e \in E^{\infty}_d.\, FGW_{d,e}])$π ⊨ G(Pd ⇒ [FDd ∨ ∃e ∈ Ed∞. FGWd,e]), completing the proof of condition (NS)(NS). □□

Appendix B. Notation

Section titled “Appendix B. Notation”

Symbol	Meaning
$\Sigma^{\mathrm{in}}, \Sigma^{\mathrm{out}}, \Sigma^{\mathrm{int}}$Σin, Σout, Σint	Disjoint input, output, and internal action sets
$\mathrm{tr}(M)$tr(M)	Finite visible execution traces of a nonempty-initialized machine
ϵε, $u \preceq t$u ⪯ t, $\mathrm{pref}L$pref L	Empty word, prefix order, prefix closure
λλ, $\lambda^* = \alpha\lambda_*$ = αλ∗	Letter-or-erasure action map and its homomorphic word extension
π_{in}πin, $Q \restriction_A$ Q ↾ A	Input projection and input-history restriction before hiding
$\mathrm{Restrict}_A(N)$RestrictA(N)	Operational restriction on state-history pairs
$T_{v,c}(Q)$Tv,c(Q)	$\lambda^*_{v,c}(Q \restriction_{A_{v,c}})$λv,c∗(Q ↾ Av,c)

Symbol	Meaning
$B_{v,c}$Bv,c, $L_{v,c}$Lv,c	Parent–owned slot monitor and its language, or a declared coarse slot predicate
$C_v[N]$Cv[N], W_vWv, K_vKv	Concrete witness composition, trajectory skeleton, monotone denotation
AdmAdm, DeliveryDelivery	Witness admission and actual scoped joint–delivery conditions
$G = (V, M, \chi, I, J, E)$G = (V , M, χ, I , J , E)	Cells, models, realization/decomposition, intents, support, evidence
AA, $A(v)$A(v)	Active intents and those owned by vv; distinct from edge input languages $A_{v,c}$Av,c
$\Phi(v)$Φ(v)	Intersection of active owned intent predicates
Deps(r)Deps(r), H_rHr	Full hypothesis dependency closure and its common–world conjunction
H_r^{safe}Hrsafe, H_r^{op}Hrop	Context premises for finite safety versus operational obligations
$\text{Beh}_{\text{exec}}(v)$Behexec(v)	Trace language of the declared scoped runtime witness
Prot(G)Prot(G)	Tagged protected intent and slot inventory, including scope
O1, O2, O3O1, O2, O3	Operational disciplines: RLNF, deterministic monitors, single–owner state
D_ℓDℓ, en_ℓenℓ, δ_ℓδℓ	RLNF leaf, enabled–input function, pure terminating handler
elab(D)elab(D), Packets(D)Packets(D)	Two–rule microstep machine and complete packet words
Sh_ℓShℓ, Trans_ℓTransℓ	Substrate shell and source–to–elaboration translation premise
FF , U_FUF	Root–covering cut and its recursively composed upper abstraction
Req(G)Req(G), σ_qσq	Typed scoped obligation set and an obligation's interpretation/context
ReuseReuse, CheckCheck	Partition of Req(G_{t+1})Req(Gt+1) into applicable evidence reuse and fresh verification
$P_d, D_d, W_{d,e}$Pd, Dd, Wd,e	Duty dd pending, duty terminal outcome emitted, genuine external wait at ee
E_d^∞Ed∞	Contract–declared set of external prerequisites permitted to wait indefinitely
$\text{CExec}_v(N_v, \omega)$CExecv(Nv, ω)	Maximal complete executions of the declared runtime semantics
ReadyReady	Operational readiness: pending duty without wait enables an owned continuation
NSNS	Contract–relative non–silent response condition under Strategy 1
H_{write}Hwrite	Explicit environmental premise guaranteeing eventual successful write response
H_{exec}^0Hexec0	Explicit per–transition weak–fairness scheduling premise for the worked example

References

Section titled "References"

- Abadi, M., and Lamport, L. 1991. The Existence of Refinement Mappings. *Theoretical Computer Science* 82(2), 253—284.
- Abadi, M., and Lamport, L. 1995. Conjoining Specifications. *ACM Transactions on Programming Languages and Systems* 17(3), 507—534.
- Abrial, J.–R. 2010. Modeling in Event–B: System and Software Engineering. Cambridge University Press.
- Bloomfield, R., Netkachova, K., and Rushby, J. 2024. Defeaters and Eliminative Argumentation in Assurance 2.0. arXiv:2405.15800.
- Cavalcanti, A. L. C., Sampaio, A. C. A., and Woodcock, J. C. P. 2003. A Refinement Strategy for Circus. *Formal Aspects of Computing* 15(2—3), 146—181.
- Clarke, E. M., Grumberg, O., Jha, S., Lu, Y., and Veith, H. 2003. Counterexample–Guided Abstraction Refinement for Symbolic Model Checking. *Journal of the ACM* 50(5), 752—794.
- Cruanes, S., Hamon, G., Owre, S., and Shankar, N. 2013. Tool Integration with the Evidential Tool Bus. *VMCAI 2013*, LNCS 7737, 275—294. Springer.
- Cruanes, S., Heymans, S., Mason, I., Owre, S., and Shankar, N. 2014. The Semantics of Datalog for the Evidential Tool Bus (Extended Abstract). *Specification, Algebra, and Software*, LNCS 8373, 256—275. Springer.

- de Alfaro, L., and Henzinger, T. A. 2001. Interface Automata. *ESEC/FSE 2001*, 109–120. ACM.
- Filipovic´, I., O'Hearn, P., Rinetzky, N., and Yang, H. 2010. Abstraction for Concurrent Objects. *Theoretical Computer Science* 411(51–52), 4379–4398.
- Green, T. J., Karvounarakis, G., and Tannen, V. 2007. Provenance Semirings. *PODS 2007*, 31–40. ACM.
- Hawblitzel, C., Howell, J., Kapritsos, M., Lorch, J. R., Parno, B., Roberts, M. L., Setty, S., and Zill, B. 2015. IronFleet: Proving Practical Distributed Systems Correct. *SOSP 2015*, 1–17. ACM. doi:10.1145/2815400.2815428.
- Herlihy, M. P., and Wing, J. M. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12(3), 463–492.
- Lamport, L. 1994. The Temporal Logic of Actions. *ACM Transactions on Programming Languages and Systems* 16(3), 872–923.
- Lynch, N. A., and Tuttle, M. R. 1989. An Introduction to Input/Output Automata. *CWI Quarterly* 2(3), 219–246.
- Mokhov, A., Mitchell, N., and Peyton Jones, S. 2018. Build Systems à la Carte. *Proceedings of the ACM on Programming Languages* 2 (ICFP), Article 79, 1–29.
- Murphy, L., Viger, T., Di Sandro, A., and Chechik, M. PLACIDUS: Engineering Product Lines of Rigorous Assurance Cases. *iFM 2024*, LNCS 15234, 87–108. Springer; first online 2024, proceedings imprint 2025. Preprint: arXiv:2407.10345 (2024).
- Murphy, L., Sy, W., Balevicius, R., Fung, K., Kung, K., Babikian, A. A., and Chechik, M. 2026. AssureCode: Structured Justification for Trustworthy Agentic Coding. arXiv:2610.29758.
- Rushby, J. 2005. An Evidential Tool Bus. *ICFEM 2005*, LNCS 3785, p. 36. Springer. Full invited paper.
- Varadarajan, S., Bloomfield, R., Rushby, J., Gupta, G., Murugesan, A., Stroud, R., Netkachova, K., and Wong, I. H. 2023. CLARISSA: Foundations, Tools & Automation for Assurance Cases. *42nd AIAA/IEEE Digital Avionics Systems Conference (DASC)*. doi:10.1109/DASC58513.2023.10311213.
- Woodcock, J., and Cavalcanti, A. 2004. A Tutorial Introduction to Designs in Unifying Theories of Programming. *iFM 2004*, LNCS 2999, 40–66. Springer.